# Text LLM Studio

User manual

November 2026

# About this manual

Text LLM Studio is the web app of this project. It runs on your own computer and lets you collect rights-cleared text and code, train writing models and coding copilots (from scratch or by fine-tuning an open model), chat with them, rate their answers and teach them from those ratings.

This manual walks through every page of the Studio in the order you would normally use them.

> **About the screenshots.** They were taken from a real Studio, but with demo data: a few MB of generated stand-in text, a 4M-parameter "tiny" model trained for a few minutes, and a tiny stand-in base model for the fine-tuned examples (shown as `models/tiny-base` instead of a Qwen model). Numbers such as loss, vocabulary size and scores therefore look different from a real run, and the example answers in Chat and Analytics were written by hand for illustration. The Copilot pages used two small demo projects and a local stand-in for GitHub serving three demo repositories (plus a GPL one and one without a licence, which were refused). Every button, field and page is exactly as the app shows them.

**Contents**


1. Installing and starting the Studio
2. Accounts and roles
3. Dashboard
4. Training: New model
5. Training: Data
6. Training: Your texts
7. Training: Retrain (a new model from one you trained)
8. Training: News & web
9. Training: Your style
10. Training: Copilot (a model that writes code)
11. Training: Corpus
12. Training: Cluster (several PCs train one model)
13. Training: Runs, run details, Resume, Export and Import
14. Chat
15. Analytics
16. Feedback
17. Users
18. Settings and hardware
19. Command-line equivalents
20. Troubleshooting

# 1. Installing and starting the Studio

You need Python 3.10 or newer. Get the project with `git clone https://github.com/hovchik/textllm` (or download the zip from GitHub) and start it with one command:

| System | Command |
|---|---|
| Windows | double-click **run.bat** (or run `run.bat` in a terminal) |
| macOS / Linux | `./run.sh` |
| Any | `python run.py` |

The first start creates a virtual environment in `.venv`, installs the packages and creates the data folders. This takes a few minutes. Then the Studio starts and your browser opens **http://127.0.0.1:7860**. Later starts are quick: packages are only reinstalled when `requirements.txt` or the PyTorch build changed. Stop the Studio with **Ctrl+C** in its window.

Useful options for `run.bat` / `run.sh` / `run.py`:

| Option | What it does |
|---|---|
| `--port 8000` | use another port (if 7860 is busy the next free port is used anyway) |
| `--host 0.0.0.0` | let other devices on your network open the Studio |
| `--no-browser` | don't open the browser |
| `--gpu` / `--xpu` / `--cpu` | force the CUDA (NVIDIA), the Intel Arc (XPU) or the CPU build of PyTorch |
| `--torch-index cu128` | pick an exact PyTorch build (also `xpu`, `rocm6.2`) |
| `--reinstall` | reinstall all packages |
| `--no-venv` | use your current Python instead of `.venv` |

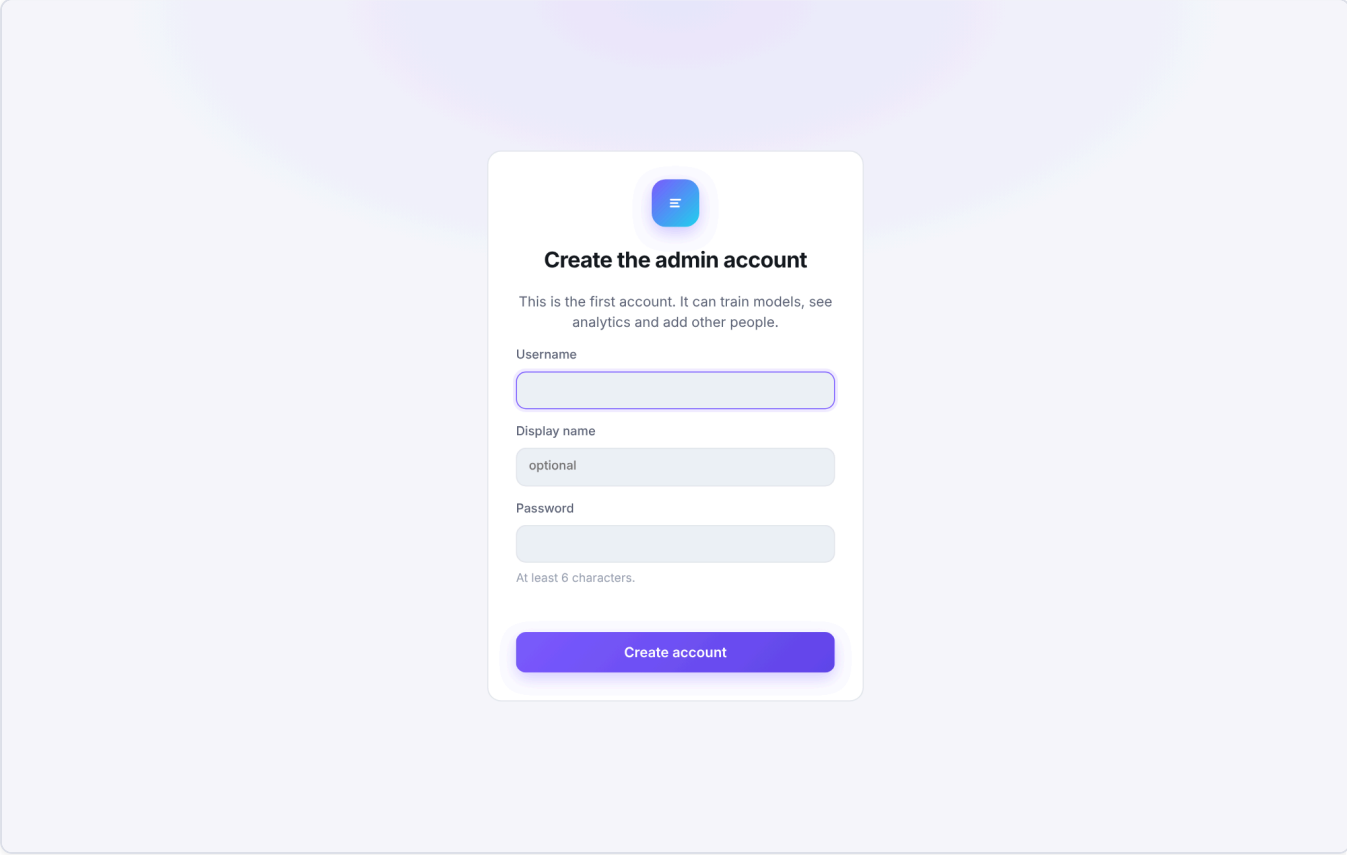
**GPU.** With an NVIDIA graphics card, `run.bat` finds it with `nvidia-smi` and installs the CUDA build of PyTorch automatically. With an **Intel Arc** card (A-series, B-series, or the Arc graphics in Core Ultra laptops) it installs PyTorch's Intel (XPU) build instead; only the Intel graphics driver is needed. Apple Silicon Macs use the Apple GPU. Everything else (including older Intel UHD / Iris Xe graphics) trains on the CPU.

On Intel Arc, from-scratch training, LoRA fine-tunes, chat and cluster workers all use the card (bf16). Not available there: 4-bit loading (bitsandbytes) and Liger kernels, which are NVIDIA-only, so LoRA keeps the base model in bf16; and GGUF chat stays on the CPU unless llama-cpp-python is built with Vulkan. Which device is used is shown on the Dashboard and can be changed under Settings → Hardware (section 18).

These installer flags are the only options that are not in the Studio itself, because they run before the Studio starts. Every other script option has a control in the web app.

# 2. Accounts and roles

The very first time you open the Studio it asks you to create the **admin** account. Choose a username and a password of at least 6 characters; the display name is optional.



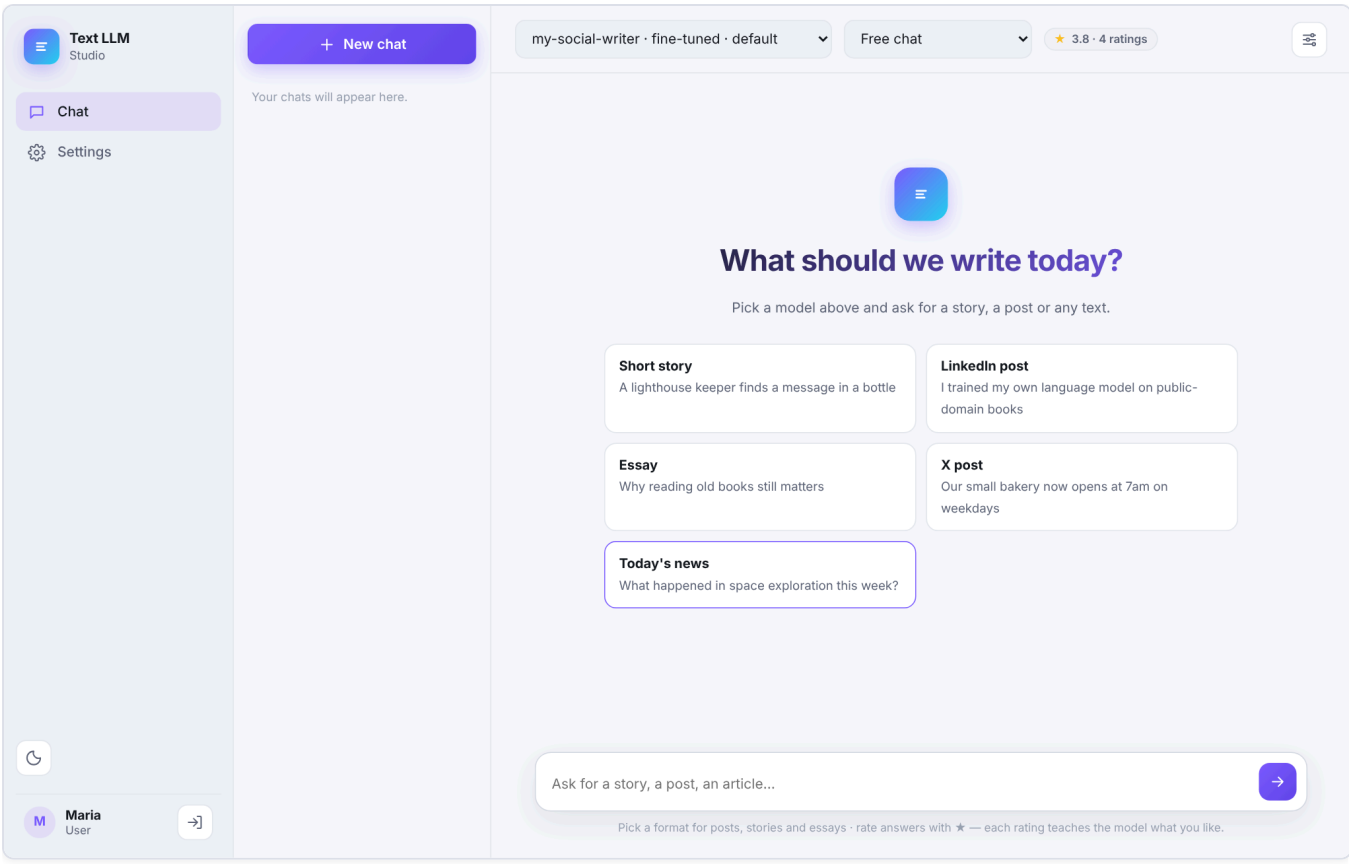

*First start: create the admin account*

After that everyone signs in with their own account. There are two roles:

| Role | Can see |
|---|---|
| **Admin** | every page: Dashboard, Training, Chat, Analytics, Feedback, Users, Settings (including Hardware) |
| **User** | only **Chat** (their own conversations) and **Settings** (name, password, chat defaults) |

A regular user's ratings and corrections still teach the models. The server checks the role on every request, so hiding a page is not the only protection. Passwords are stored as salted hashes in `runs/studio.db`.

This is what a regular user sees: only Chat and Settings in the sidebar.

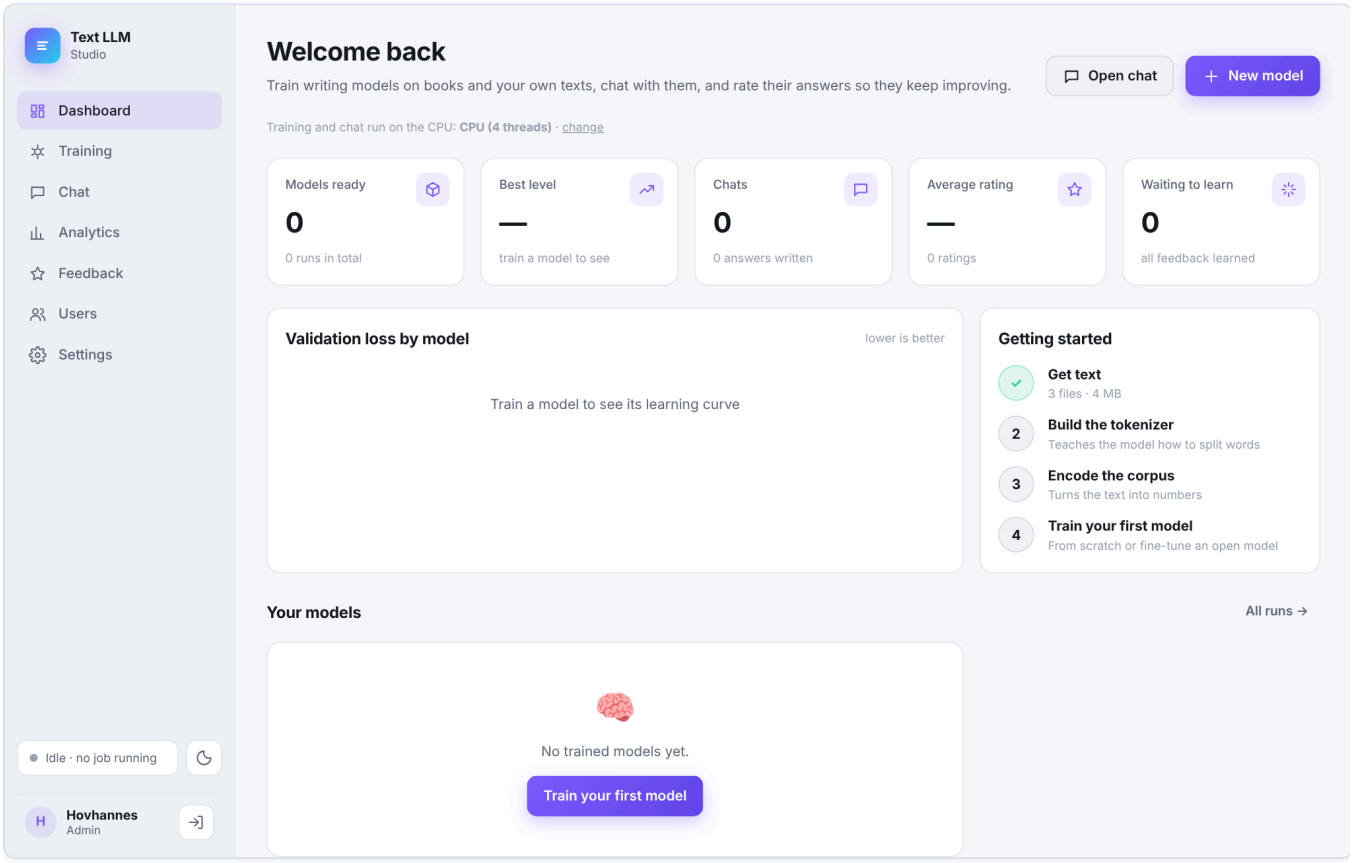



*A regular user only sees Chat and Settings*

The bottom of the sidebar shows who is signed in, a **sign out** button, a **theme** button (light / dark) and, for admins, the **job pill**. The job pill says “Idle · no job running” or names the job that is running; click it to jump to Training → Runs.

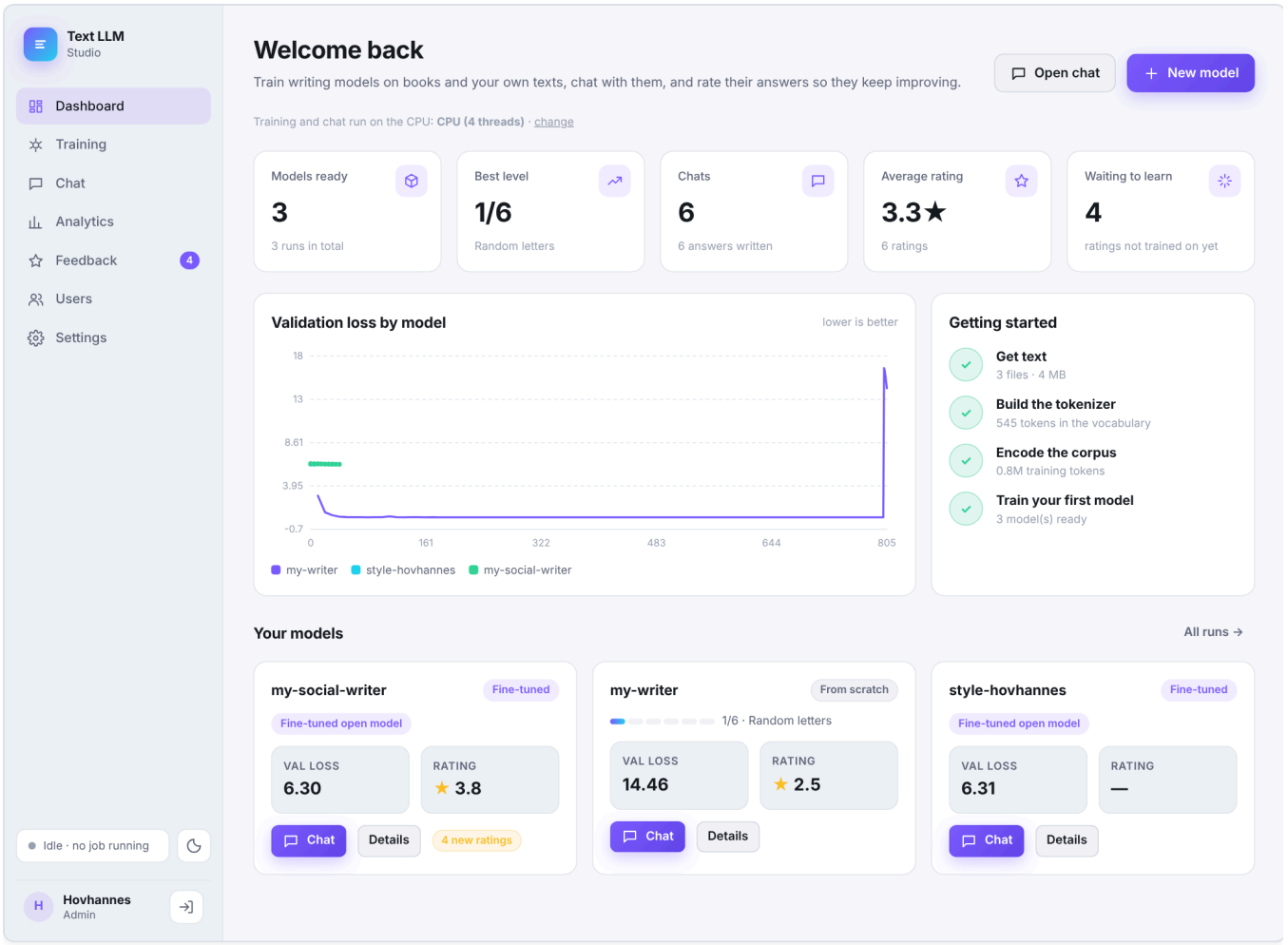
# 3. Dashboard

The Dashboard is the home page for admins.




*Dashboard on a fresh install*

On a fresh install the **Getting started** list shows what to do next: get text, build the tokenizer, encode the corpus, train your first model. Each step is a link to the right page and gets a green check mark when it is done.




*Dashboard with three models*

Once you have models it shows:

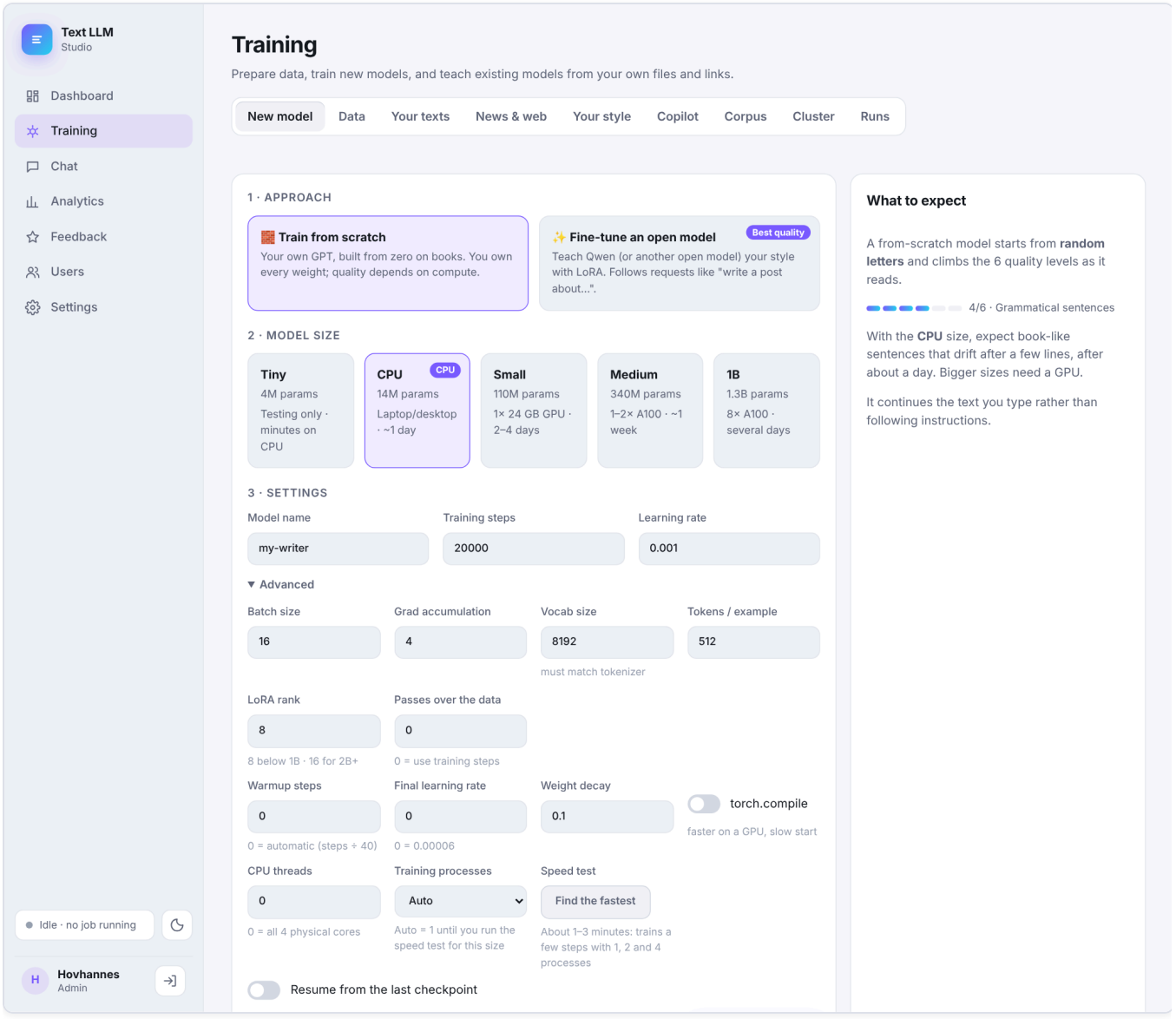
- **Device line:** whether training and chat run on the GPU or the CPU, with a **change** link to Settings.
- **KPIs:** models ready, best level reached by a from-scratch model (1/6 to 6/6), number of chats and answers, average star rating, and ratings that are **waiting to learn**.
- **Validation loss by model:** learning curves of all models (lower is better).
- **Your models:** a card per model with its type (From scratch / Fine-tuned), level, validation loss and average rating, plus **Chat** and **Details** buttons. A yellow “N new ratings” tag means the model has feedback it hasn’t learned yet.
- **Live card:** while a job runs, a card with a progress ring, step, ETA, loss and a **Stop** button appears at the top.
- **Open chat** and **New model** buttons at the top right.

**Note.** For a from-scratch model the “validation loss” after **Your texts** or feedback learning measures how well it knows the *new* text, so the number and the curve jump up at that point. That is expected, not a broken model.

# 4. Training: New model

**Training** has seven tabs: New model, Data, Your texts, News & web, Your style, Corpus and Runs. **New model** is a three-step wizard.

## 4.1 Train from scratch




*New model: train from scratch, with Advanced open*

1. **Approach:** *Train from scratch* builds your own GPT from zero. You own every weight; quality depends on how much compute you give it. It continues text rather than following instructions.
2. **Model size:**

| Size | Parameters | Hardware |
|---|---|---|
| Tiny | 4M | testing only, minutes on CPU |
| CPU | 14M | laptop or desktop, about a day |
| Small | 110M | one 24 GB GPU, 2–4 days |
| Medium | 340M | 1–2 × A100, about a week |
| 1B | 1.3B | 8 × A100, several days |

3. **Settings:** model name, training steps (default 20,000) and learning rate (0.001).

**Advanced** (click to open): batch size, gradient accumulation, vocabulary size (must match the tokenizer), tokens per example, warmup steps (0 = steps ÷ 40), final learning rate, weight decay, **torch.compile** (faster on a GPU, slow start) and **Resume from the last checkpoint**.

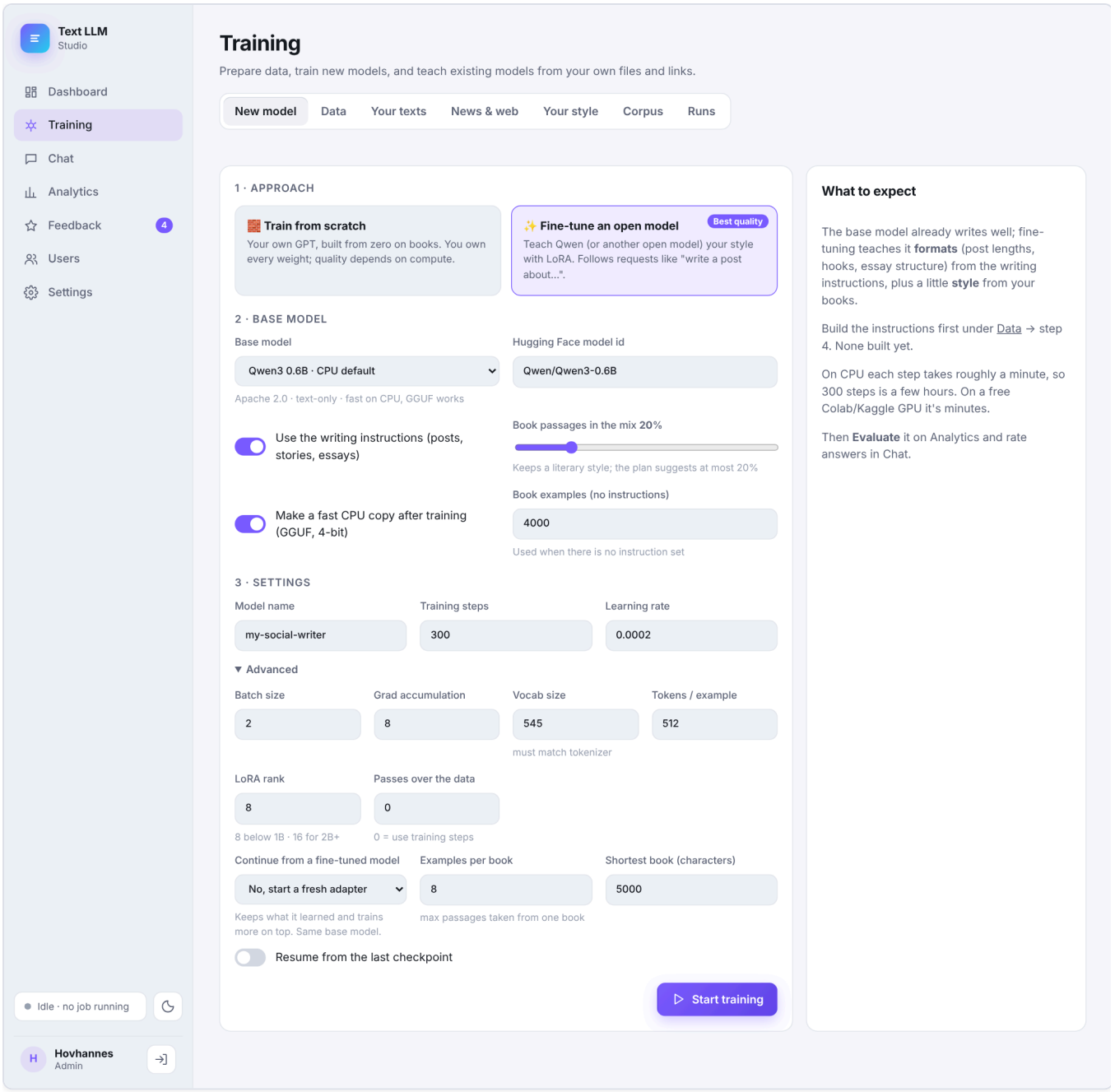
**Speed on this PC** (also under Advanced):

- **CPU threads**: how many CPU threads training uses. 0 (the default) means all physical cores, which is what training always did. Set a lower number to keep the PC usable for other work while it trains. LoRA fine-tuning uses this setting too.
- **GPU start-up limit (min)**: on a GPU the first step also compiles the GPU's kernels, which on Intel Arc (especially the Arc graphics built into Core Ultra laptops) can take several minutes; the log says so and prints the time of step 1. If step 1 takes longer than this limit (default 20 minutes), the run restarts on the CPU by itself. 0 waits as long as it takes.
- **Training processes**: from-scratch models can train in several processes on one PC. Each process takes part of the batch and a share of the threads, and after every step they average their gradients, so the model learns the same thing as one process with the whole batch. **Auto** uses one process until you run the speed test, and then the count the test found fastest for that model size. On a PC with several NVIDIA GPUs, Auto starts one process per GPU.
- **Find the fastest**: a speed test that takes 1 to 3 minutes. It trains a few steps of the chosen model size on random tokens with 1, 2 and 4 processes and remembers the fastest. A count only wins if it is at least 5% faster than one process. The result shows under Training processes, for example "Auto = 2 for this size (1 → 900 ms/step, 2 → 700 ms/step)".

Whether several processes help depends on the CPU. PyTorch already spreads the big matrix multiplies of one process over every core, so on a 4-core test machine 1, 2 and 4 processes ran within 5% of each other (CPU size, about 1.25 s per step with batch 16 × 2). CPUs with many cores, or with hyper-threads, are more likely to gain, which is why the speed test measures your own PC instead of guessing. Threads and processes belong to the PC, not to the run: **Resume** uses the values in the form, so a run moved to another PC uses that PC's settings, and a run trained with 2 processes can continue with 1.

Before training from scratch you must run Data steps 1–3 (section 5). The right-hand card "What to expect" explains the six quality levels: random letters → … → grammatical sentences → stays on topic.

## 4.2 Fine-tune an open model (LoRA)




*New model: fine-tune an open model*

Choose **Fine-tune an open model** to teach an existing open model (Apache 2.0) your formats and style with LoRA. This gives the best writing quality and the model follows requests like “write a post about...”.

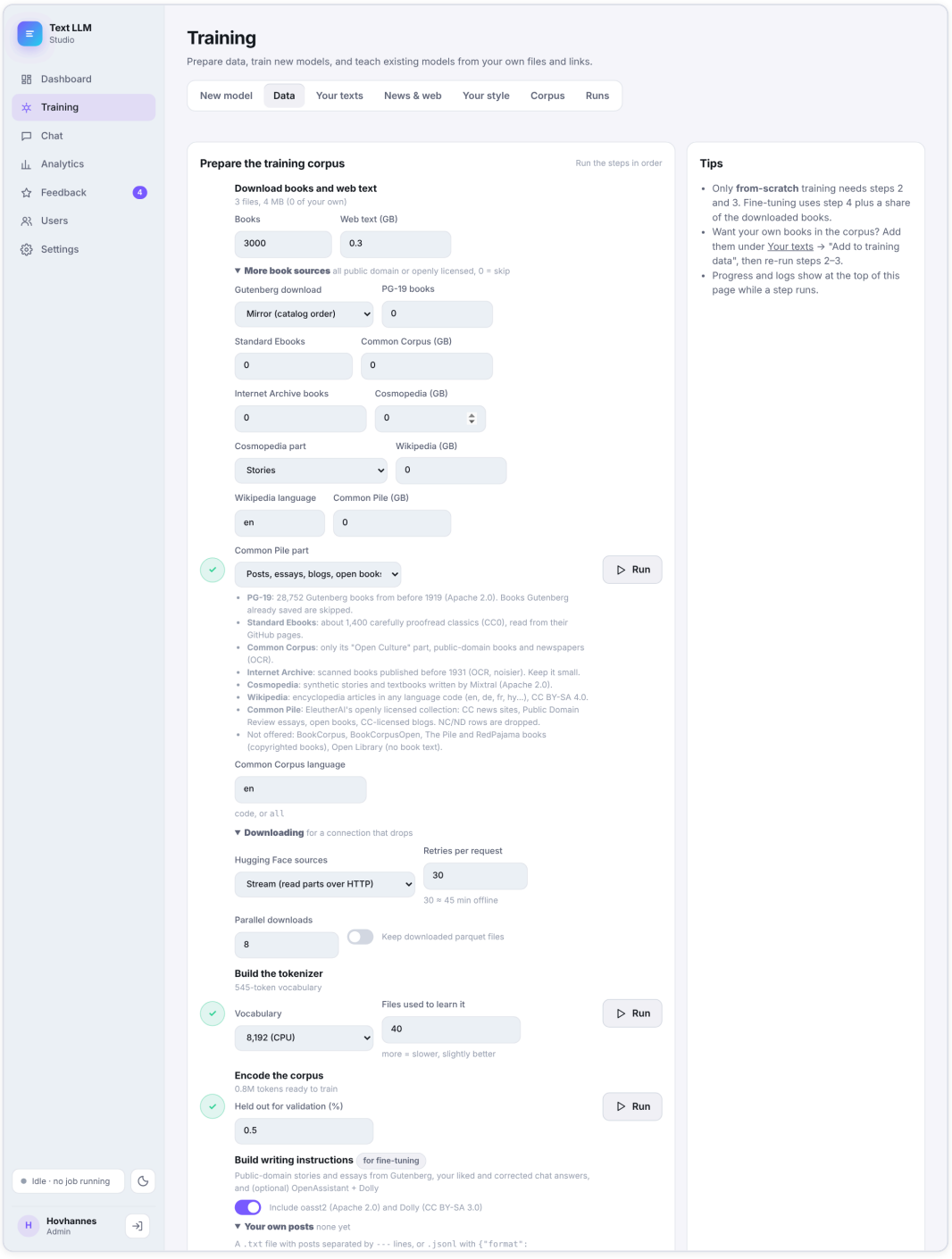
- **Base model:** Qwen3 0.6B (CPU default), Qwen3 1.7B (better writing), Qwen3.5 0.8B / 2B (newest, experimental), SmolLM3 3B, Qwen2.5 0.5B, or **Other** to type any Hugging Face model id.
- **Use the writing instructions (posts, stories, essays):** trains on the set built in Data → step 4.
- **Book passages in the mix:** share of book continuation (default 20%) so the model keeps a literary style.
- **Make a fast CPU copy after training (GGUF, 4-bit):** makes chat much faster on a CPU.
- **Book examples:** used only when there is no instruction set.

Defaults switch to 300 steps, learning rate 0.0002, batch 2, accumulation 8. Under **Advanced** you also get **LoRA rank** (8 below 1B, 16 for 2B+), **passes over the data** (0 = use training steps), **Continue from a fine-tuned model** (keeps what it learned, same base model), **examples per book** and **shortest book**.

On a CPU each fine-tuning step takes about a minute, so 300 steps is a few hours. Press **Start training**; the Studio opens the run's details so you can watch it.

# 5. Training: Data

The Data tab prepares the training corpus in four steps. Run them in order with each step's **Run** button. A step with a green check is done; the log of the running step is shown at the bottom of the page.



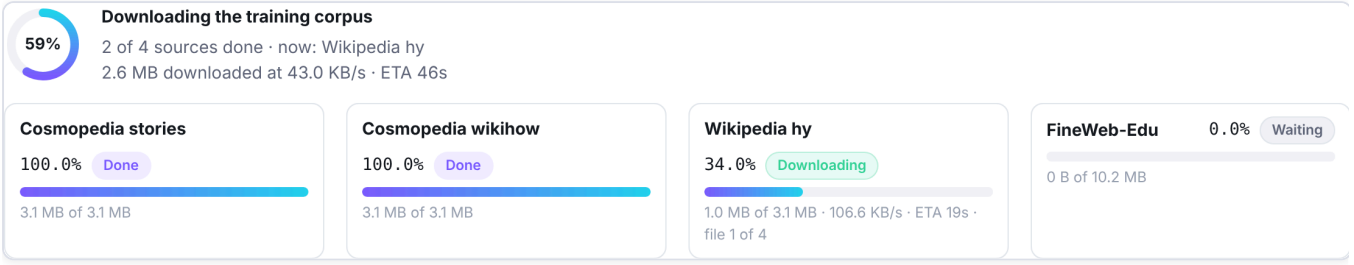

*Data: the four preparation steps with all options open*

**Step 1 · Download books and web text.** *Books* is how many Project Gutenberg books to download (default 3,000); *Web text (GB)* is how much FineWeb-Edu (openly licensed educational web text) to add.

- **More book sources** (all public domain or openly licensed; 0 = skip): Gutenberg download method (mirror or official robot harvest), PG-19 books, Standard Ebooks, Common Corpus (GB and language), Internet Archive books from before 1931, Cosmopedia (GB and part: stories, how-to guides, textbooks…), Wikipedia (GB and language code, e.g. `en`, `hy`), Common Pile (GB and part: posts and essays, CC news, CC blogs, open books, Wikibooks/Wikivoyage, spoken style, public-domain OCR books).
- **Downloading** (for a connection that drops): stream Hugging Face sources or download whole files first (resumable), retries per request (30 ≈ 45 minutes offline), parallel downloads, keep downloaded parquet files.
- Not offered on purpose: BookCorpus, The Pile, RedPajama books (copyrighted books) and Open Library (no book text).

If the download stops, press **Run** again: every source continues where it stopped.

**Download progress.** While a download runs, the *Progress* card (above the log) shows every source you chose, in the order they are fetched, refreshed every few seconds:




*Download progress: overall ring, then one card per source*

- **Overall:** the ring is the share of all chosen sources that is done, then how many sources are finished, which one is downloading now, how much came over the network (and how fast), and the time left for the whole run.
- **Each source:** *Waiting*, *Downloading*, *Done*, *Skipped* (nothing to do, e.g. already downloaded) or *Stopped*/*Failed*; a bar with the percent; how much text is saved out of what you asked for (books, or MB/GB of text); its speed and time left; and for Hugging Face sources which file it is reading ("file 2 of 14", plus how much of that file has arrived in *Download whole files first* mode).
- Speeds and times left follow the last minute, so they settle after a minute and adjust when the connection slows. Text saved can grow faster than the network figure, because Hugging Face files are compressed.
- The Dashboard's running-job card shows the same overall percent and time left. After the job ends, the card keeps the last run's numbers.

The same progress shows on the command line: `python data/build_corpus.py ...` prints a bar per source and an `[overall 40% · 2/5 sources done · 1.2 GB downloaded · ETA 35m]` line before each source and at the end.

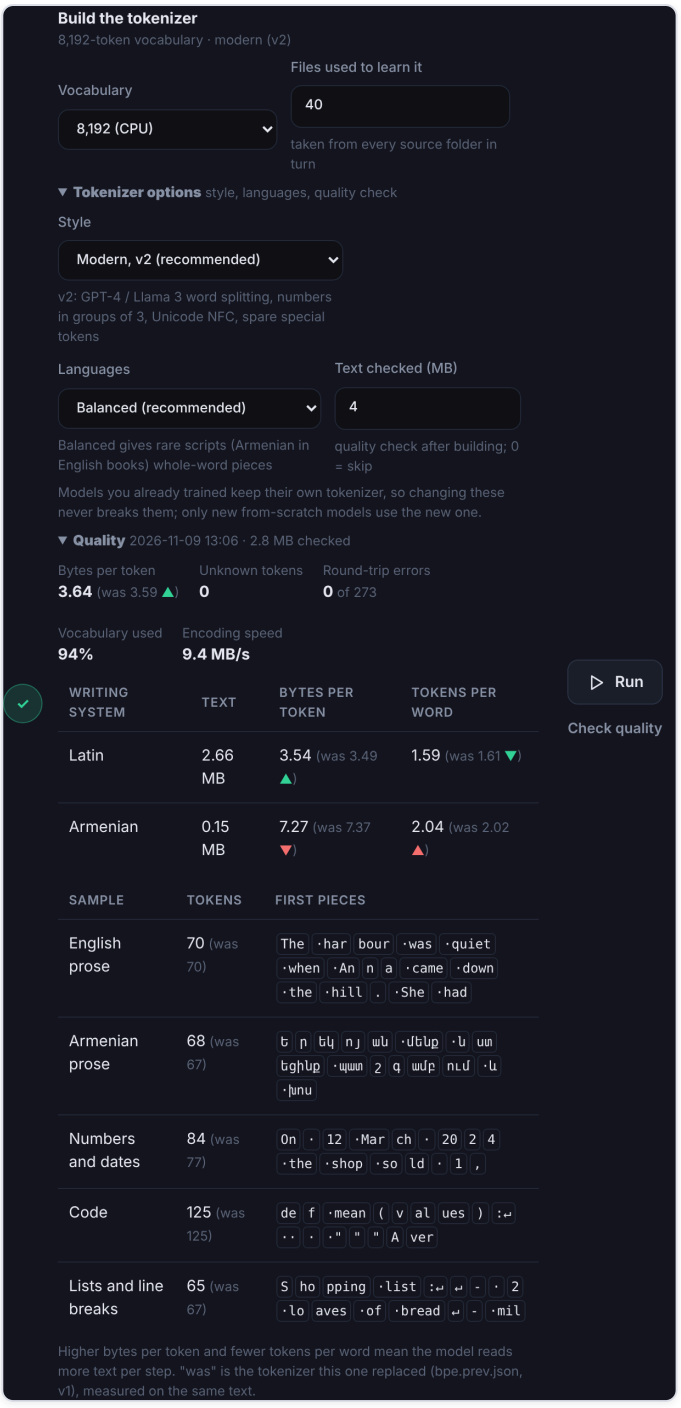
**Step 2 · Build the tokenizer.** Learns the word pieces the model reads and writes. Vocabulary 8,192 for CPU models or 32,000 for GPU models; *Files used to learn it* (more is slower and slightly better). The files are taken from every source folder in turn (`gutenberg`, `fineweb`, `custom`, …), so your own texts

are always part of it. While it runs, the *Progress* card shows three steps: *Reading text* (how many MB of the chosen files are read, speed and time left), *Learning word pieces* (no percent, usually a few minutes) and *Checking quality*.

**Tokenizer options** (click to open):

- **Style.** *Modern, v2* (the default) works like the tokenizers of GPT-4, Llama 3 and Qwen: Unicode NFC first (the same letter is always stored the same way), word splitting that keeps numbers in groups of up to 3 digits, handles `DON'T` like `don't` and keeps line breaks and indentation as their own pieces, and 16 special tokens (end of text, padding, chat turns and spare slots, so a later chat fine-tune needs no resize). *GPT-2 style, v1* is the tokenizer every model before November 2026 used. Both are byte-level: any text, Armenian, emoji or code, is encoded with no “unknown” token and decodes back exactly.
- **Languages.** *Balanced* (default) reads text in rarely seen writing systems (Armenian among English books, for example) several times while learning, up to 8 times, so their words get whole pieces instead of single letters. *As in the corpus* uses the text as it is.
- **Text checked (MB).** How much of the corpus the quality check reads after building (0 = skip).

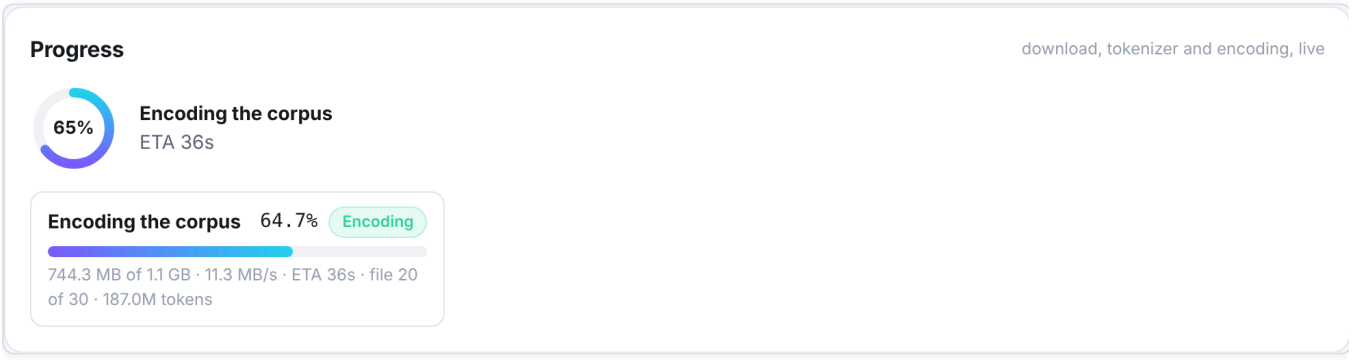
Changing these never breaks a model you already trained: every model keeps the tokenizer it was trained with (`runs/<name>/tokenizer.json`), and its version shows under the model’s name in Training → Runs (“tokenizer v1” or “v2”). The replaced tokenizer is kept as `tokenizer/bpe.prev.json`.




*Data step 2: tokenizer options and the quality check*

**Quality** (under step 2, after building or after **Check quality**): bytes per token over the checked text (higher = the model reads more text per step; English books are about 3.5 to 4.5), unknown tokens (always 0), round-trip errors (text that does not decode back; always 0 unless something is broken), how much of the vocabulary is used, and encoding speed. A table per writing system shows bytes per token and tokens per word; Armenian should be near English, not 3 to 10 times higher. The *Sample* table shows how fixed sentences (English, Armenian, numbers, code, line breaks) are cut into pieces. After a rebuild, every number also shows what the previous tokenizer had on the same text (*was* ...).

**Step 3 · Encode the corpus.** Turns all text into token files for fast training. *Held out for validation* is the share kept aside to measure validation loss (default 0.5%). While it runs, the *Progress* card and the Dashboard's running-job card show the percent of the corpus encoded, MB done of the total, speed, which file it is on, how many tokens so far and the time left:




*Encoding the corpus: percent, MB, speed, tokens and time left*

Encoding uses every CPU core and works through the text a few MB at a time, so it needs well under 1 GB of memory whatever the size of the corpus. Expect roughly 5 to 15 MB of text per second on a 4-core laptop, so 10 GB of text takes about 15 to 30 minutes. On the command line `python tokenizer/train_tokenizer.py --encode` prints a bar with MB done, speed and time left.

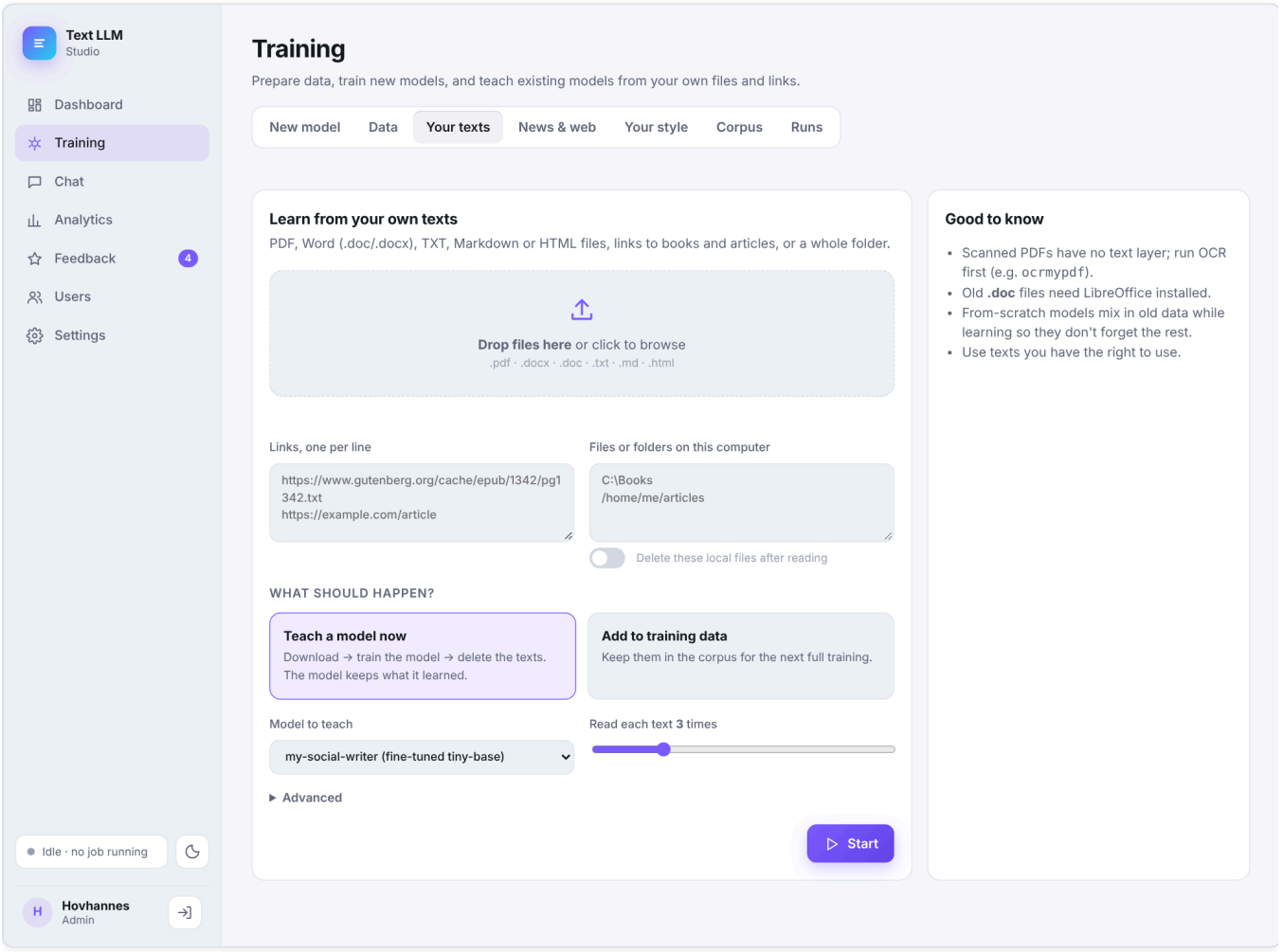
Steps 2 and 3 are only needed for from-scratch training. After you add or delete text, run **step 3 (Encode) only** if you want to keep training an existing model: a model must always read text encoded with the tokenizer it was trained with, and a new tokenizer gives the same token numbers different meanings without any error. Build a new tokenizer (step 2, then step 3) only before training a **new** model from scratch.

**Step 4 · Build writing instructions** (for fine-tuning). Makes examples of posts, stories and essays from public-domain Gutenberg stories and essays and your liked and corrected chat answers, plus optionally OpenAssistant (Apache 2.0) and Dolly (CC BY-SA 3.0).

- **Your own posts:** upload `.txt` files (posts separated by `---` lines) or `.jsonl` files (`{"format": "linkedin_post", "text": "..."}`). Only your own posts.
- **More options:** number of Gutenberg stories and essays, books to read per kind, random seed, a **model that writes example posts** (synthetic posts per format; use a GPU) and a **model that writes the requests** (back-translation).

# 6. Training: Your texts

Teach a model your own books, articles and notes, or add them to the corpus.



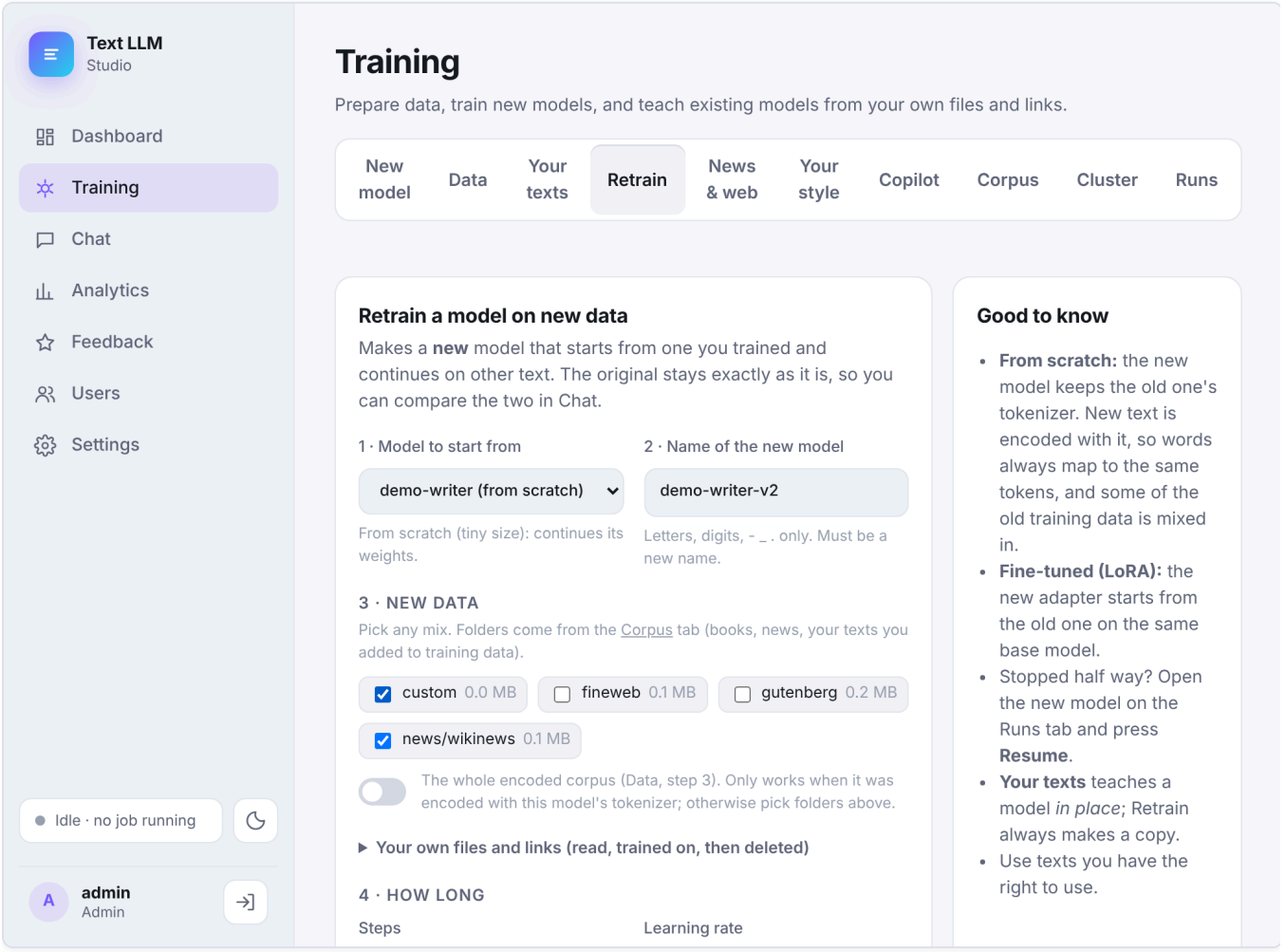

*Your texts*

1. **Add texts:** drop files onto the box (or click to browse): PDF, Word (.docx and .doc), TXT, Markdown and HTML. You can also paste **links**, one per line (articles, online PDFs, plain-text books), and **files or folders on this computer** (folders are searched recursively). Tick *Delete these local files after reading* if you want them removed.
2. **What should happen?**
    - **Teach a model now:** download → train the chosen model → delete the extracted text. The model keeps what it learned. Choose the **model to teach** and how many times to **read each text** (1–10, default 3). Under **Advanced**: how much old text is mixed in per new token (from-scratch models only, so they don't forget; default 1) and the learning rate (0 = automatic).
    - **Add to training data:** keeps them in `data/raw/custom/` for the next full training. Then run Data step 3 (Encode); also step 2 only if you will train a new model from scratch (see section 5).
3. Press **Start**.

Good to know: scanned PDFs (pictures of pages) are read with OCR when Tesseract is installed; the *Scanned PDFs (OCR)* section shows whether it is and lets you pick the languages (e.g. `eng+hye`). A file that can't be read (password-protected, not really a PDF, empty) is skipped with a message saying why, and the others still go ahead. Old `.doc` files need LibreOffice; only use texts you have the right to use.

# 7. Training: Retrain (a new model from one you trained)

Continue a model you already trained on **other data**, as a **new model**. The original is never changed, so you can compare the two in Chat and keep whichever writes better. Use it when you have collected new books or news, built a new corpus, or want a version of your writer that leans towards one kind of text.




*Retrain*

1. **Model to start from:** any trained model, from scratch or fine-tuned (LoRA), writer or copilot. The hint under it says which kind it is. You can also open a model on the Runs tab and press **Retrain on new data**, which opens this tab with that model chosen.
2. **Name of the new model:** a new name (it suggests `<name>—v2`). An existing model's name is refused, so nothing is ever overwritten.
3. **New data**, any mix of:
   - **Corpus folders:** tick the folders to learn from (the same list as the Corpus tab: `gutenberg`, `fineweb`, `custom`, each news source, ...). The text is encoded with the **model's own tokenizer**, so word-to-token mapping always matches the model, even if you rebuilt `tokenizer/bpe.json` since.

    - **The whole encoded corpus** (from-scratch models only): uses `data/bin` from Data step 3 as it is. This only works when it was encoded with the same tokenizer as the model; otherwise the Studio refuses and tells you to pick folders instead (they get re-encoded with the model's tokenizer).
    - **Your own files and links:** like the Your texts tab. They are read, trained on and then deleted; because of that, such a retrain can't be resumed after a stop.
4. **How long: Steps** (default 2000 from scratch, 200 fine-tuned; 0 = read the new data once) and the **learning rate** (0 = automatic: 0.0003 from scratch, 0.0001 fine-tuned; lower keeps more of the original model).
5. Under **Advanced: old data mixed in per new token** (from scratch, default 0.5: some of what the model was trained on is mixed into the new data so it does not forget everything else; 0 = only the new data), batch and gradient accumulation, and for fine-tuned models whether to make the fast CPU copy afterwards.
6. Press **Start retraining**. The new model's window opens with its own loss curve. Its run details say *retrained from <name>*.

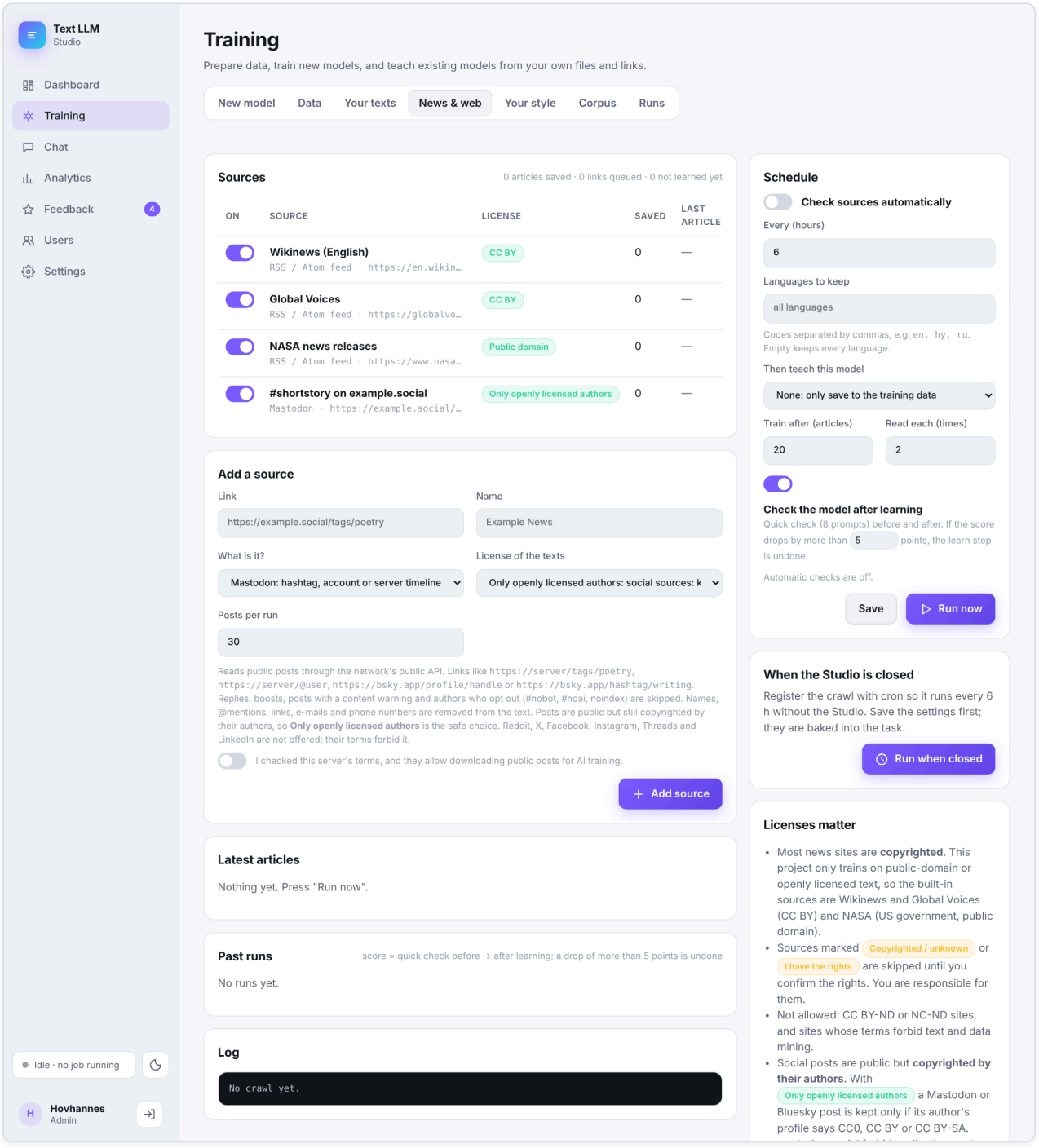
What happens underneath:

- **From scratch:** the new model starts from the original's weights and tokenizer, with a fresh learning-rate schedule (short warm-up, then down to a tenth). The old training data is mixed in only when it was encoded with the same tokenizer. The encoded new data is kept in `runs/<new>/retrain-data/`, so **Resume** works after a stop.
- **Fine-tuned (LoRA):** the new adapter starts from the original's adapter on the same base model and rank, and learns continuation examples cut from the new text (code examples for a copilot). The writing style profile (Your style) carries over.
- **Your texts → Teach a model now** changes a model *in place*; **Retrain** always makes a copy.

From a terminal: `python retrain.py --from runs/my-writer --out runs/my-writer-news --source data/raw/news --steps 2000` (section 19).

# 8. Training: News & web

Reads RSS/Atom feeds, web pages and public social posts on a schedule, saves new articles to `data/raw/news/<source>/`, and can teach a model the new articles once enough have arrived.



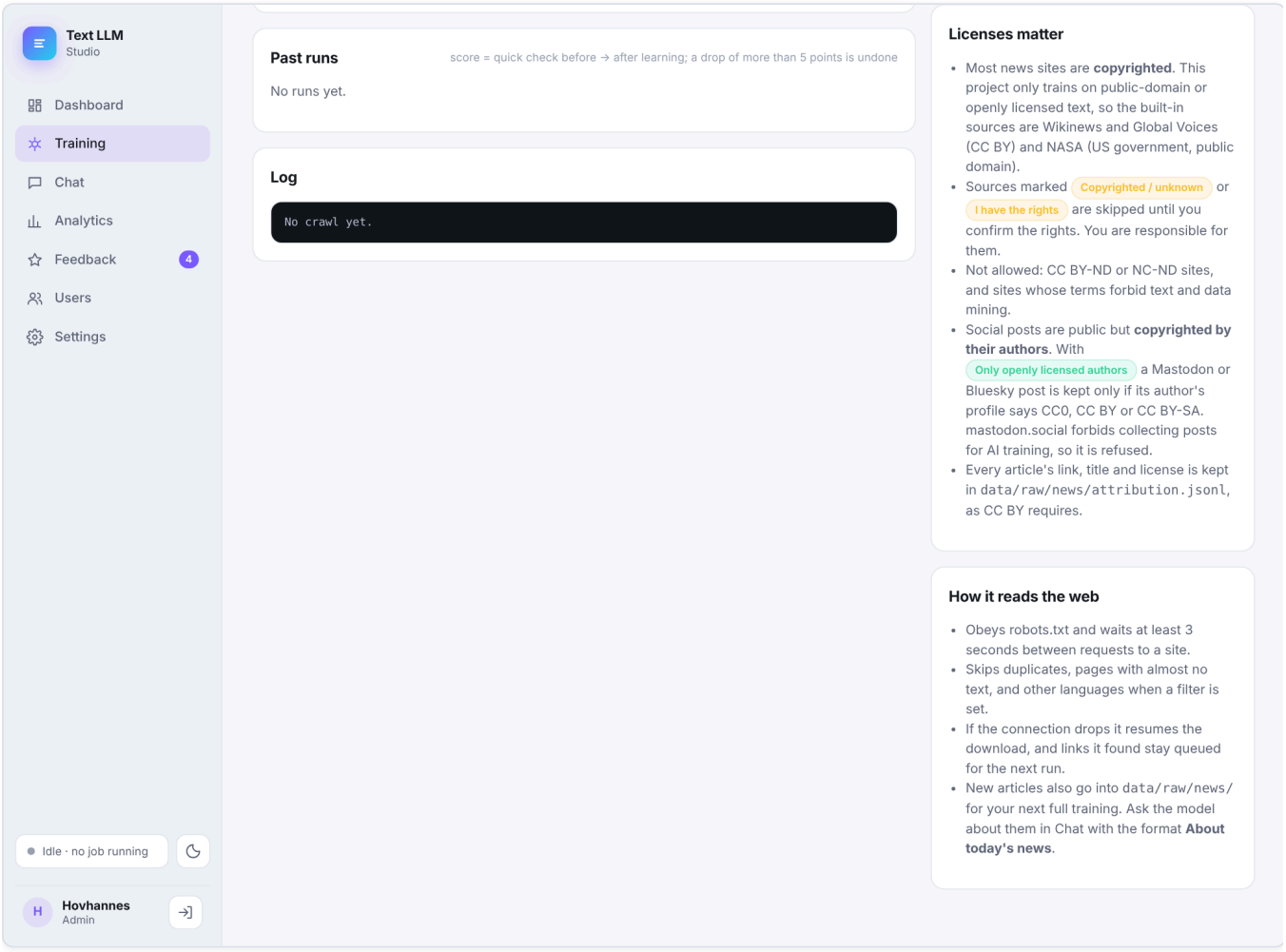

*News & web: sources, adding a Mastodon source*

**Sources.** The table lists every source with an on/off switch, its kind and link, its license, how many articles were saved, when the last one arrived, and a delete button. The built-in sources are Wikinews and Global Voices (CC BY) and NASA news releases (public domain).

**Add a source.** Enter the link and a name, then pick:

- **What is it?** RSS / Atom feed (best), page of links (reads the articles it links to), single page, **Mastodon** (hashtag `https://server/tags/poetry`, account `https://server/@user` or local timeline `https://server/public/local`) or **Bluesky** (profile, custom feed, hashtag or search).
- **License of the texts.** Sources marked *copyrighted / unknown* or *I have the rights* are skipped until you tick the box that you own them or have permission. CC BY-ND and NC-ND sites are not allowed.
- **Articles (or posts) per run.**

For Mastodon and Bluesky you must also confirm that the server's terms allow downloading public posts for AI training. With the license **Only openly licensed authors** a post is kept only if its author's profile says CC0, CC BY or CC BY-SA. Replies, boosts, posts with a content warning and authors who opt out (#nobot, #noai, noindex) are skipped; names, @mentions, links, e-mails and phone numbers are removed. mastodon.social and mastodon.online forbid collecting posts for AI training, so they are refused. Reddit, X, Facebook, Instagram, Threads, LinkedIn and TikTok are not offered because their terms forbid it.




*News & web: latest articles, past runs, log and the schedule*

**Schedule** (right column):

- **Check sources automatically** every N hours.
- **Languages to keep:** codes like `en, hy, ru`; empty keeps every language.

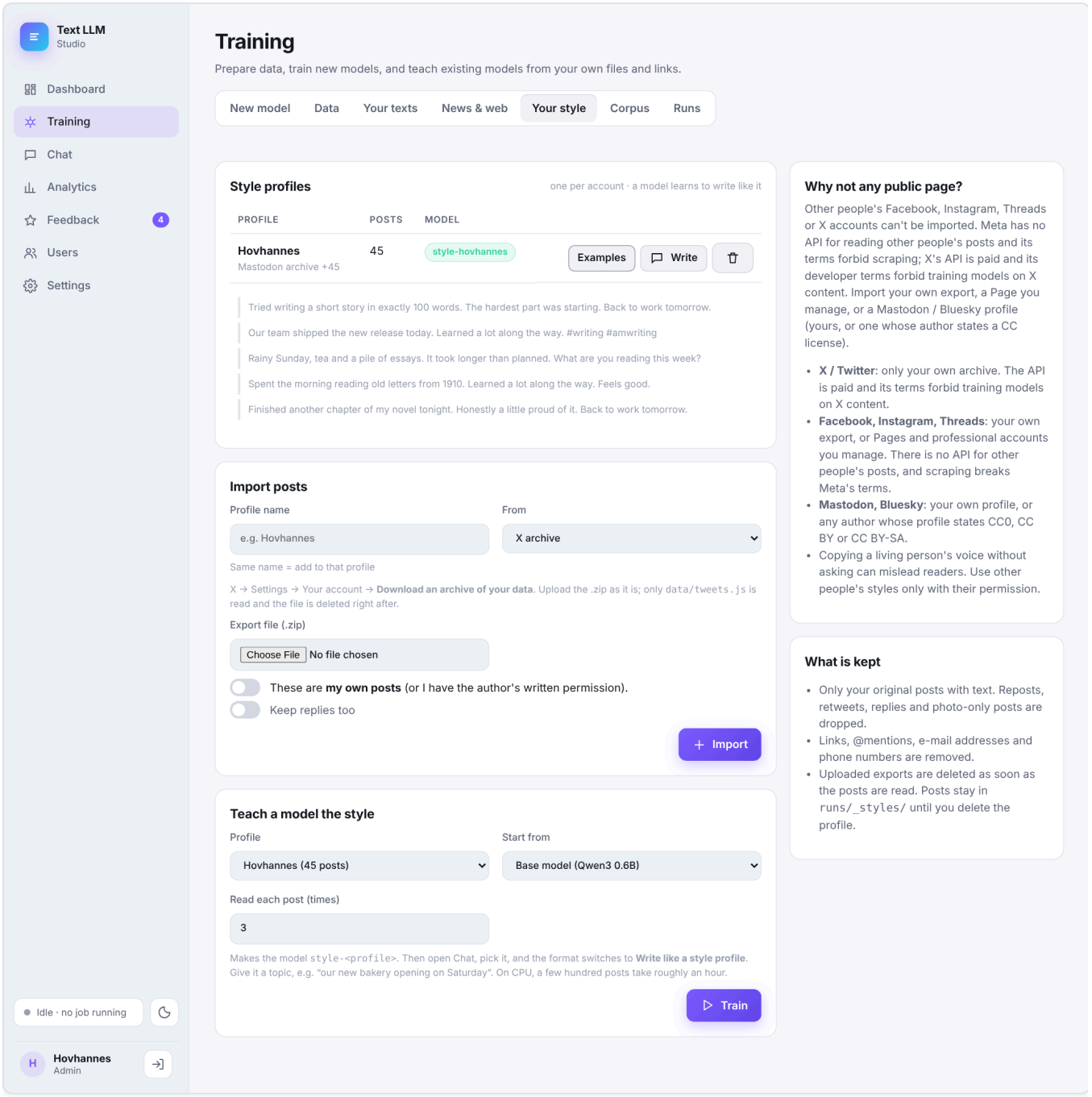
- **Then teach this model:** pick a model, or *None* to only save the articles. **Train after** N articles, **read each** N times.
- **Check the model after learning:** runs a quick check (8 prompts) before and after; if the score drops by more than the given number of points, the learning is undone.
- **Save** stores the settings; **Run now** reads all sources once.
- **When the Studio is closed: Run when closed** registers the crawl with Windows Task Scheduler (or cron on Linux/macOS) so it runs even when the Studio is not open. **Remove** / **Update** change that task.

Below the sources you see the **latest articles** (with language), **past runs** (articles added, failed, and whether learning was kept or undone with the score before → after) and the **log**.

The crawler obeys robots.txt, waits at least 3 seconds between requests to a site, skips duplicates and near-empty pages, and resumes after a dropped connection. Every article's link, title and license is written to `data/raw/news/attribution.jsonl`. In Chat, the format **About today's news** answers from the latest collected articles.

# 9. Training: Your style

Teach a model to write like one account (yours, or someone who gave permission or uses an open license).




*Your style: a profile with 45 posts and its trained model*

1. **Import posts.** Type a **profile name** (the same name adds to an existing profile) and choose where the posts come from:
    - **X archive:** X → Settings → Your account → Download an archive of your data. Upload the .zip.
    - **Facebook export / Instagram export:** "Download your information", format **JSON**. Upload the .zip.

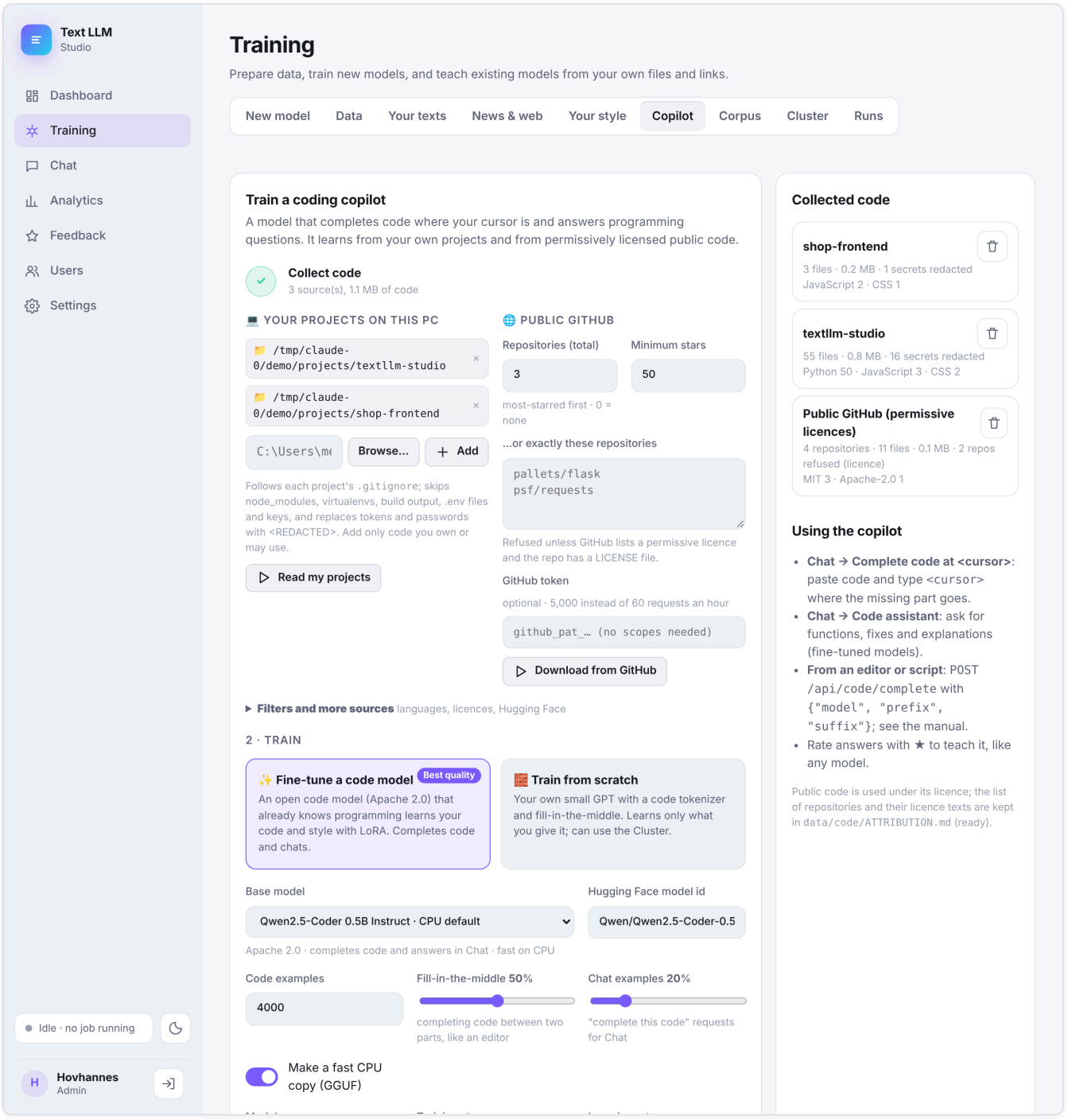
    - **Mastodon archive:** Preferences → Import and export → Request your archive.
    - **Facebook Page** or **Instagram professional account you manage:** enter the Page or account ID and a Graph API access token (used once, not stored).
    - **Mastodon or Bluesky profile link:** yours, or one whose author states CC0, CC BY or CC BY-SA.

  Tick **These are my own posts** when they are; optionally **keep replies**. Press **Import**. Uploaded files are deleted right after reading; reposts, replies and photo-only posts are dropped; links, @mentions, e-mails and phone numbers are removed.
2. **Teach a model the style.** Choose the profile (it needs at least 30 posts), **Start from** (the base model, or one of your fine-tuned models so it keeps its writing skills) and how many times to **read each post**. Press **Train**. This makes the model `style—<profile>`; a few hundred posts take about an hour on a CPU.
3. **Write.** In the profile table press **Write** (or pick `style—<profile>` in Chat). The format switches to **Write like a style profile**; give it a topic.

**Examples** shows a few imported posts; the trash button deletes the profile (and, if you agree, its model). Other people's Facebook, Instagram, Threads or X accounts can't be imported: there is no API for them and their terms forbid scraping.

# 10. Training: Copilot (a model that writes code)

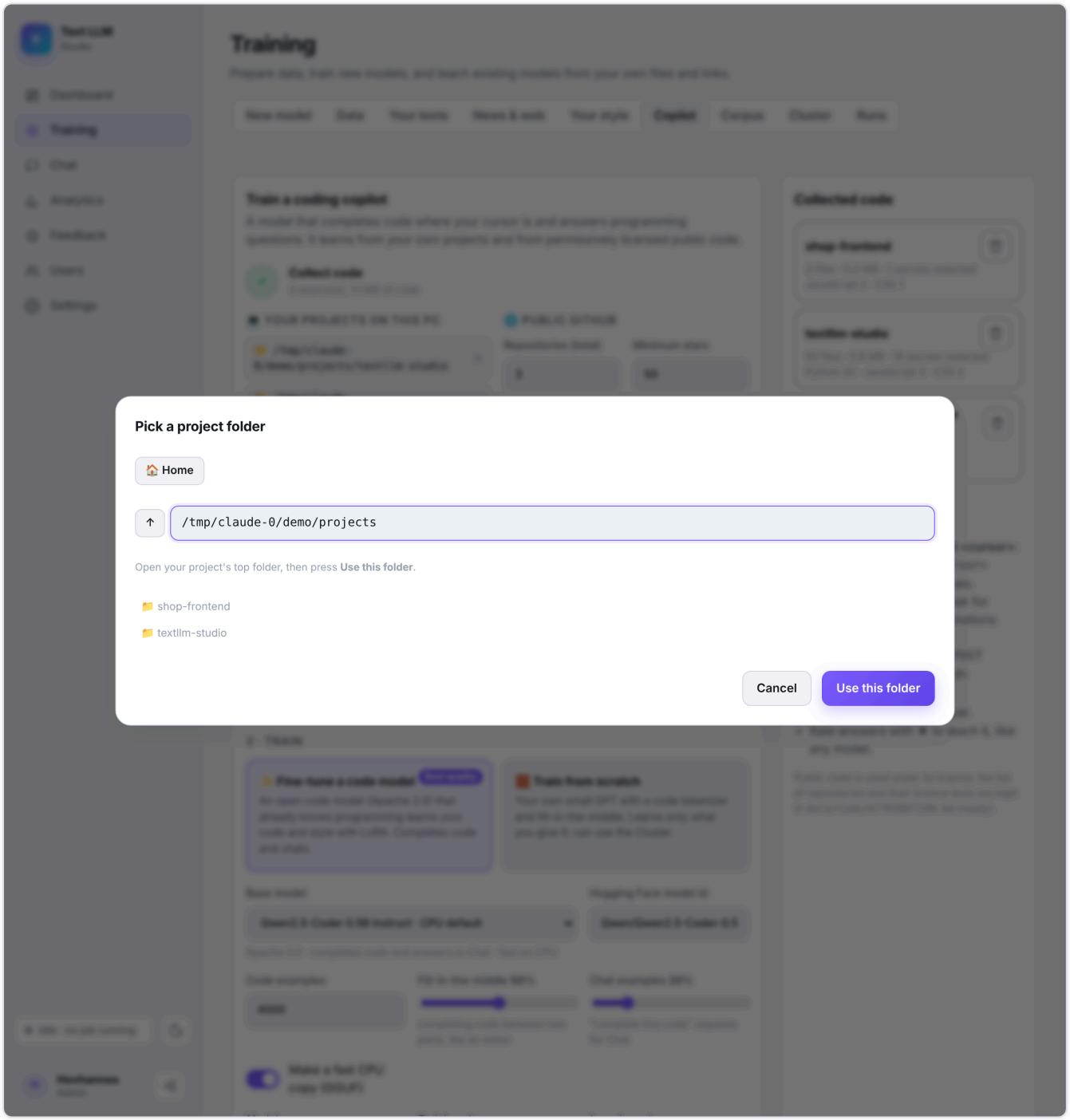
A **copilot** is a model that completes code where your cursor is and answers programming questions. The **Copilot** tab collects code from your own projects and from permissively licensed public code, and trains either a fine-tuned code model or a from-scratch one.




*Training → Copilot: code sources, collected code and training options*

## 10.1 Collect code

**Your projects on this PC.** Type a folder, or press **Browse...** to pick one, then **Add**. Add each project's top folder (the one with `package.json`, `pyproject.toml` or `.git`). Press **Read my projects** to read them all. Reading a folder again replaces its earlier copy, so press it again after you have changed your code.




*Browse... picks a project folder on this PC*

While it reads, the Studio:

- follows each project's `.gitignore` files (and `.git/info/exclude`);
- skips `node_modules`, virtualenvs, `vendor`, build output (`dist`, `build`, `target`, `bin`, `obj`), caches and IDE folders;

- never reads files that hold secrets (`.env`, `*.pem`, `*.key`, `id_rsa`, keystores, cloud credential files, `*.tfstate`…);
- replaces tokens (GitHub, AWS, Google, Stripe, OpenAI, Anthropic, Hugging Face, Slack, JWT…) and `password = "..."`-style values with `<REDACTED>`, and drops any file that contains a private key;
- skips binary, minified, generated ("do not edit"), very small and very large files, and duplicates.

This is careful but not perfect: a model can remember what it trains on, so add only code you own or may use, and that you are happy to have inside the model.

**Public GitHub.** Set how many repositories you want (most-starred first, per language) and the minimum stars, or list exact repositories as `owner/name`. A repository is used only when GitHub reports a **permissive licence** (MIT, Apache 2.0, BSD, ISC, Unlicense, CC0, zlib, Boost, 0BSD, MIT-0) **and** it has a LICENSE file. Copyleft licences (GPL, LGPL, AGPL, MPL, EPL), "other" licences and repositories with no licence are refused and listed in the log, because public code without a licence is still all rights reserved. Each repository's licence text, address and branch are saved in `data/code/github/`, and `data/code/ATTRIBUTION.md` lists them all; keep that file with any model you share. A **GitHub token** (any personal token, no permissions needed) raises GitHub's limit from 60 to 5,000 requests an hour; it is saved on this PC and never written to the log. Downloads resume: press the button again and finished repositories are skipped. While code downloads, the *Download progress* card above the log shows each part (chosen repositories, new commits, the GitHub search, Hugging Face code) with how many repositories (or MB) are done, the repository being downloaded, the bytes downloaded and the time left.

**Filters and more sources:** the languages to keep (Python, JavaScript and TypeScript by default; "Every programming language" or your own list such as `python,go,rust`), the largest file and repository, the licences you accept (you can only narrow the permissive list), and **Hugging Face code**: GB of the `codeparrot/github-code-clean` dataset, keeping only rows whose licence column is permissive. **Collect everything** runs all sources at once.

The **Collected code** card lists each source with files, size, languages, the licences found and how many secrets were redacted. The trash button deletes that collected copy (your project folders are never touched).

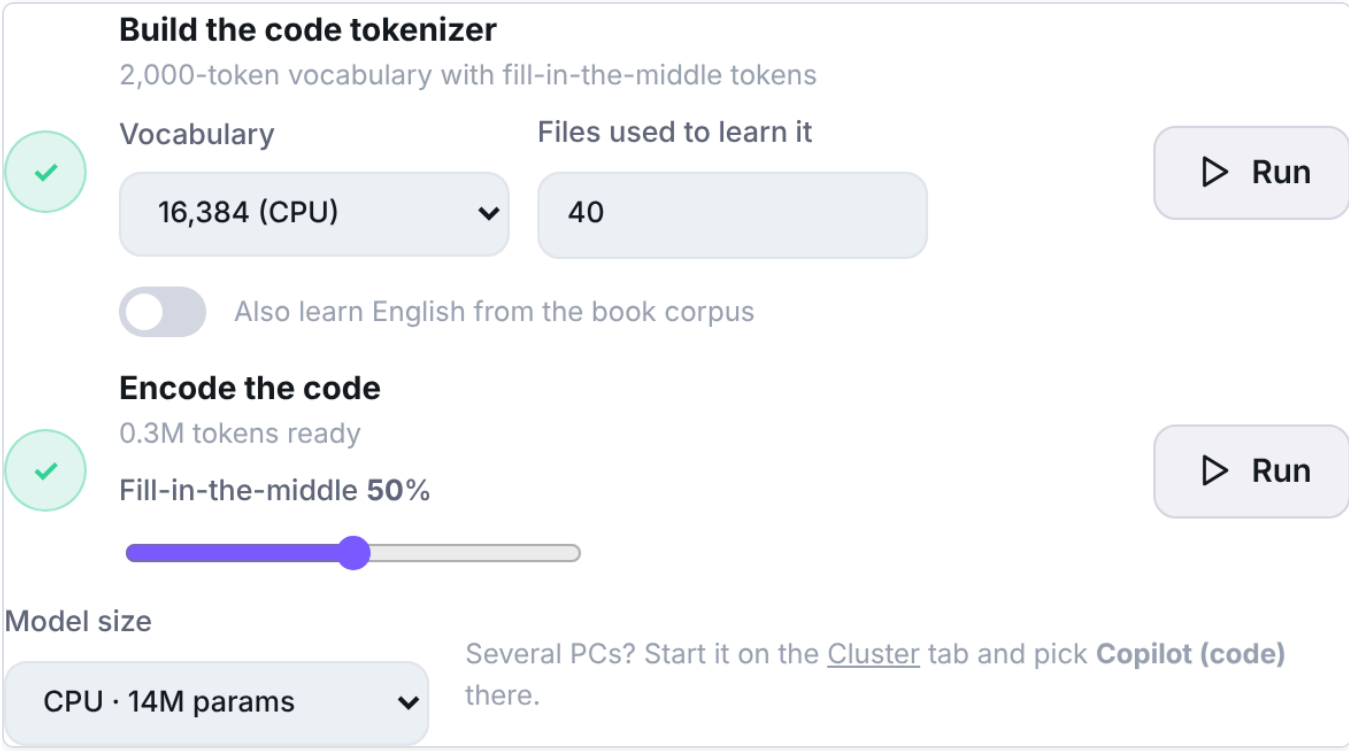
## 10.2 Train

**Fine-tune a code model** (best quality). Pick a base model: **Qwen2.5-Coder 0.5B Instruct** (default, Apache 2.0, fine on a CPU), 1.5B Instruct (better, overnight on a CPU), or a *base* Qwen2.5-Coder for completion only. The 3B model has a non-commercial licence, as its hint says. The examples are cut from your collected code:

- **Fill-in-the-middle** (50% by default): the code before and after a gap, and the missing part, exactly like an editor asking for a completion;
- **continuation**: the code so far, and the next lines;
- **Chat examples** (20%): "Complete this Python code from app/models.py…" with the *Code assistant* instructions, so the model also helps in Chat.

Set **Code examples**, the name, steps and learning rate, and press **Start training**. Advanced: batch size, gradient accumulation, tokens per example (1024; code needs more room than posts), LoRA rank, passes over the data, examples per file, only some languages, and **Continue from** an earlier copilot.

**Train from scratch.** Your own small GPT that learns only from the collected code. It needs two preparation steps here, separate from the writing model's Data steps, so the two never overwrite each other:




*From-scratch copilot: code tokenizer and encoding*

a. **Build the code tokenizer** (`tokenizer/bpe-code.json`), with the fill-in-the-middle tokens. *Also learn English from the book corpus* mixes in `data/raw` so comments and docstrings read better. *Style* is the same choice as in Data step 2 (Modern v2 recommended).
b. **Encode the code** (`data/bin-code/`). The fill-in-the-middle share of files is rearranged as before / after / missing part.

Then pick a model size and press **Start training**. To train it on several PCs, use the **Cluster** tab and choose **Kind of model: Copilot (code)** (section 12).

Copilots appear under Runs and in Chat as "copilot". They can be rated, improved, exported and resumed like any other model. (Evaluate on Analytics scores writing formats, so it says little about a copilot.)

## 10.3 Using the copilot

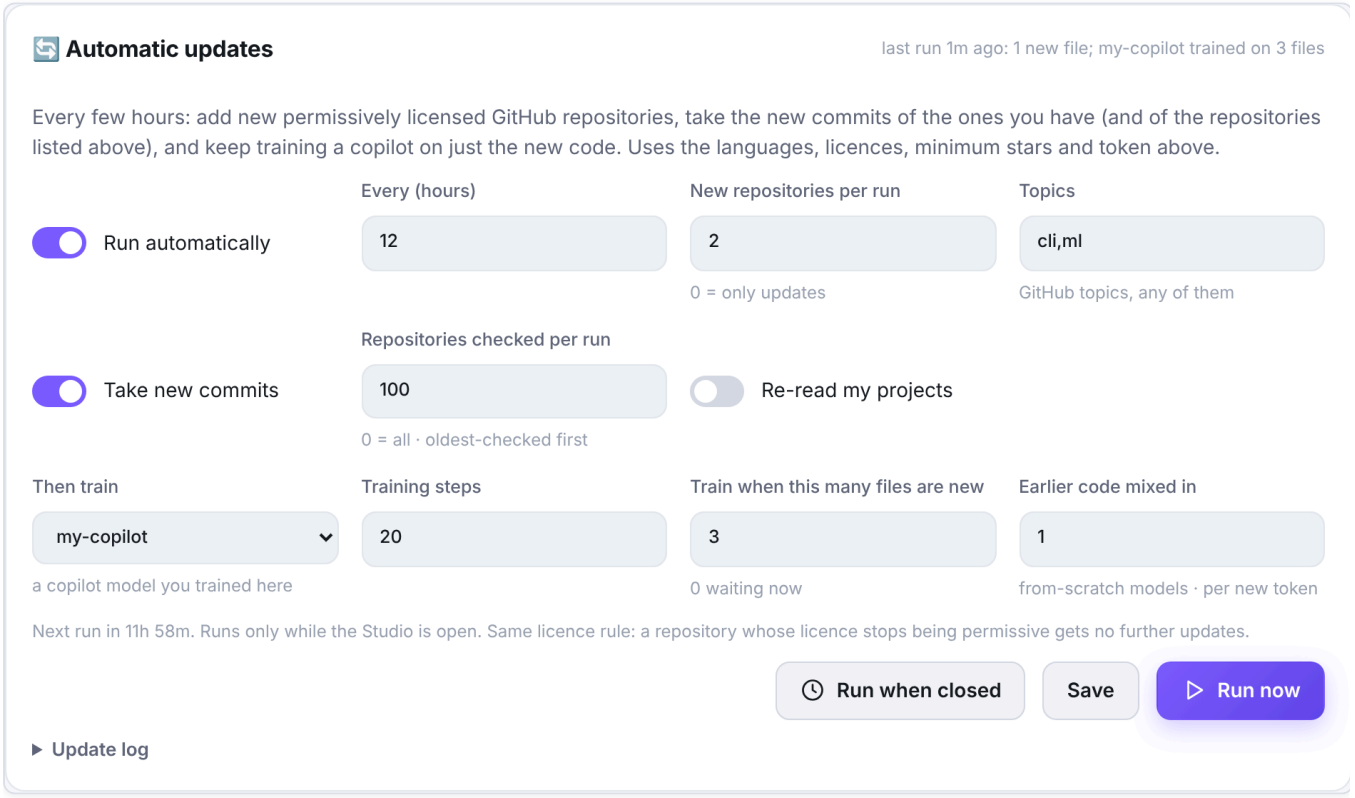
- **Chat → Complete code at <cursor>:** paste code and type `<cursor>` where the missing part goes; without it the model continues the end. The answer is the code for that spot (section 14).
- **Chat → Code assistant:** ask for a function, a fix or an explanation (fine-tuned copilots).

- **From an editor or a script:** send the code around the cursor to the Studio. Any signed-in user may call it with their session cookie:

```
POST http://127.0.0.1:7860/api/code/complete
{"model": "my-copilot", "prefix": "def add(a, b):\n    return ", "suffix": "\n",
"max_tokens": 64}
→ {"completion": "a + b", "model": "my-copilot"}      (an example answer)
```

## 10.4 Automatic updates

The **Automatic updates** card, under the training options, keeps the copilot up to date with new public code without you pressing anything.




*Automatic updates: new repositories, new commits and training on the new code*

Each run does three things:

1. **New repositories.** It adds up to **New repositories per run** public repositories, using the languages, licences, minimum stars and token from the GitHub section above. **Topics** narrows the search to GitHub topics such as `cli,django,react`; a repository needs any one of them. When every search has been read to the end, the next run starts the searches again from the top, so newly popular repositories are found too.
2. **New commits.** With **Take new commits** on, it asks GitHub whether each repository you already have was pushed since it was downloaded. If so, it downloads it again and adds only the files that are new or changed (duplicates are skipped as always). The repositories you listed under … *or exactly these repositories* are your **watchlist**: they are checked first on every run. **Repositories checked per run** caps the number of GitHub requests (without a token GitHub allows 60 an hour); the ones checked longest ago go first.

3. **Your projects.** With **Re-read my projects** on, your project folders are read again and their changed files are added too.

The licence rule stays the same. A repository whose licence stops being permissive, or whose LICENSE file disappears, gets no further updates. The code taken while it was permissive stays, and `ATTRIBUTION.md` keeps the licence each download had.

**Then train.** Choose a copilot you trained here, and the code that is new since its last training is queued. When at least **Train when this many files are new** files are waiting, the copilot trains on just those files for **Training steps** steps:

- a fine-tuned copilot continues its LoRA adapter, with fill-in-the-middle, continuation and chat examples cut from the new files;
- a from-scratch copilot continues from its checkpoint with its own tokenizer. **Earlier code mixed in** adds that many tokens of `data/bin-code` per new token, so it doesn't forget what it learned before.

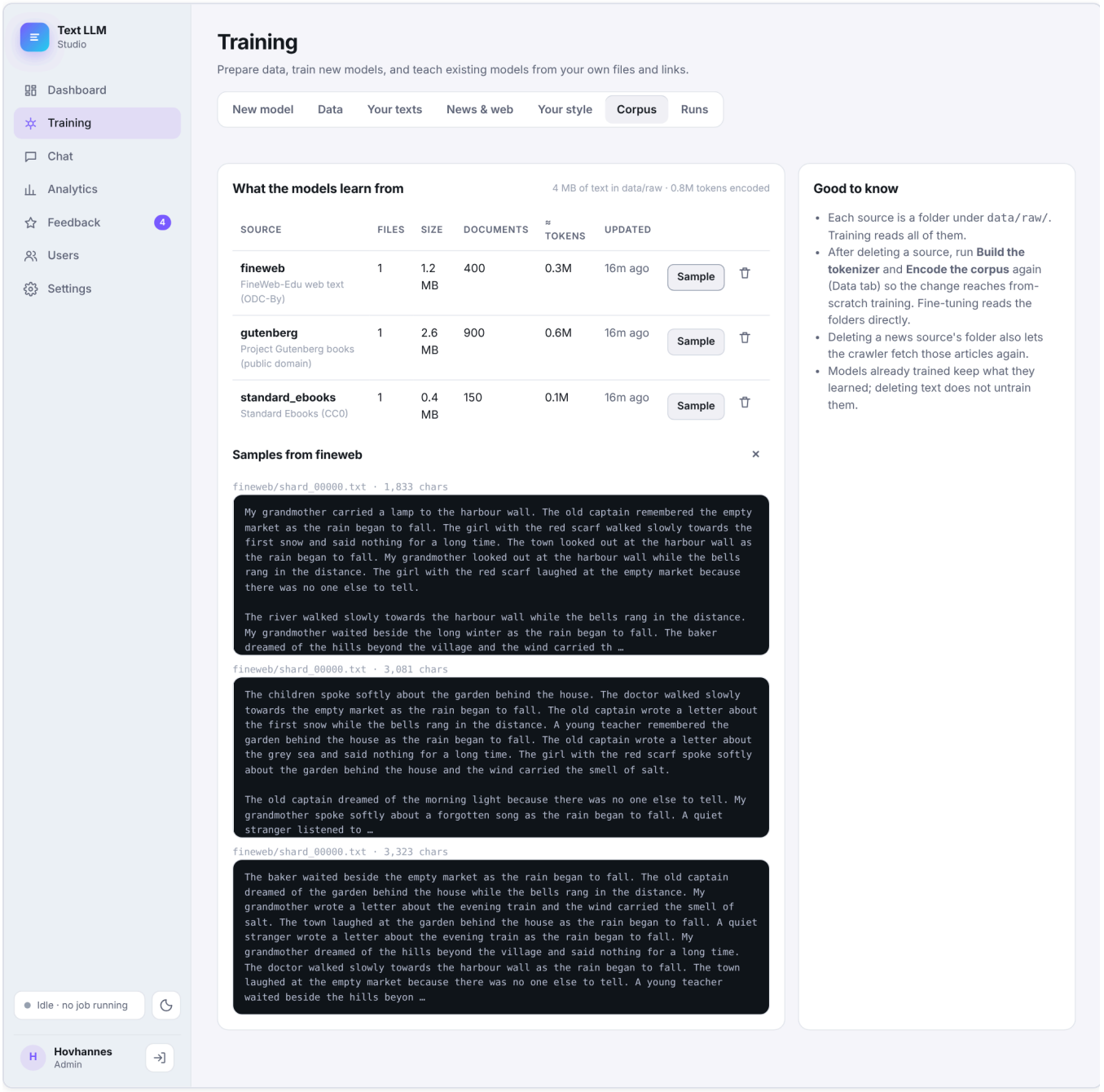
Choose *No, only collect the code* to only grow the collection, for a bigger training run later.

**When it runs.** Switch on **Run automatically** and set **Every (hours)**. The Studio then runs it while it is open, waiting if another job is busy. **Run when closed** registers it with Windows Task Scheduler (or cron on Linux and macOS), so it also runs when the Studio is closed. The task always uses the settings saved here, so press **Update** only after changing the interval. **Run now** starts one run at once. The card shows the last run and its result, how many files are waiting, and the update log. A scheduled run that finds the Studio busy skips that turn.

From the command line: `python data/code_update.py` (one run with the saved settings), `--install-schedule 24` and `--uninstall-schedule`. The building blocks are also in `data/build_code.py`: `--github-new N`, `--topics cli,web`, `--refresh` and `--refresh-limit N`.

# 11. Training: Corpus

Shows what the models learn from: one row per folder under `data/raw/`.



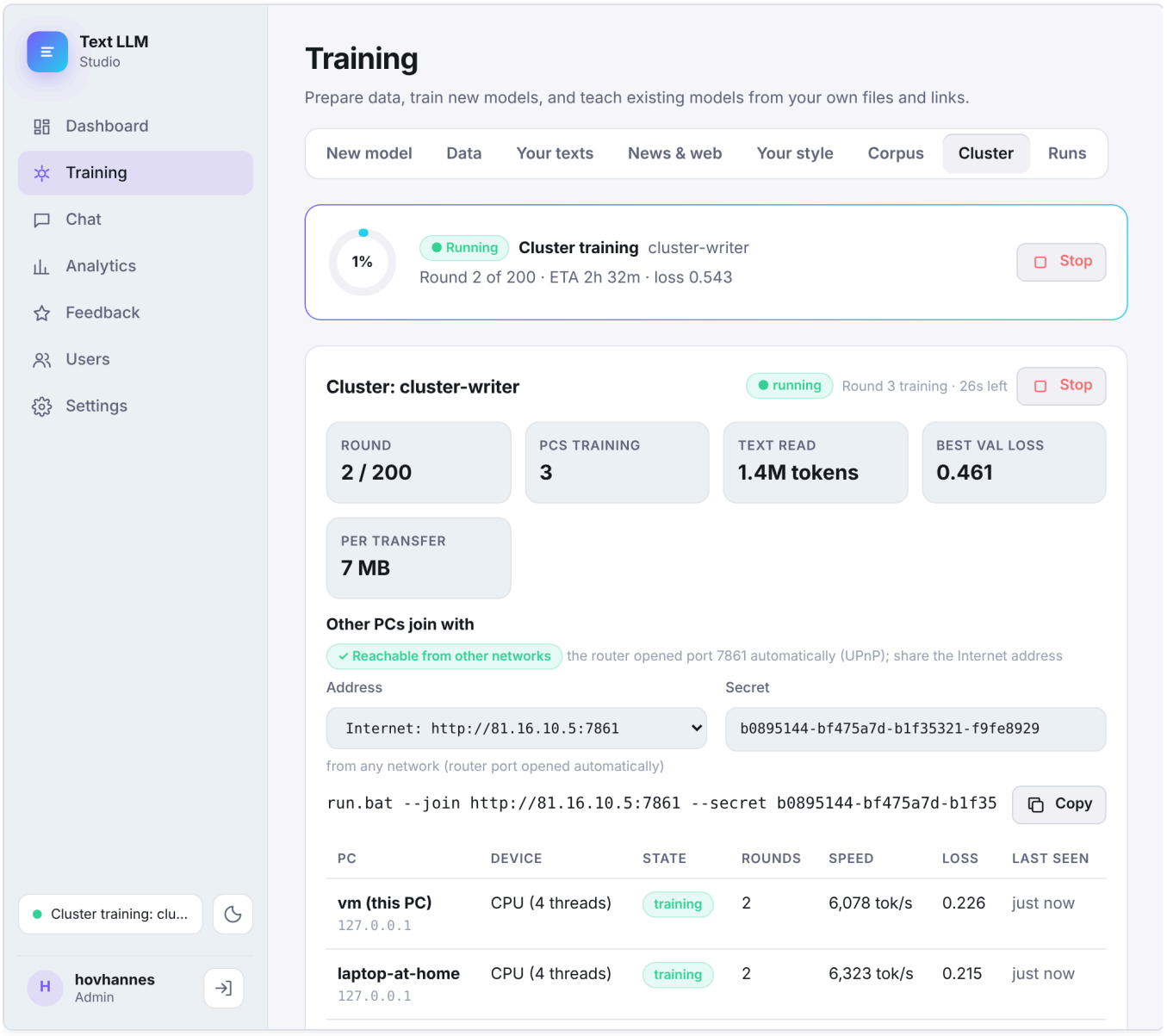

*Corpus with a sample opened*

Each row shows files, size, documents, estimated tokens and when it was updated. **Sample** shows three random documents so you can check the quality. The trash button deletes the folder (it asks first and can't be undone).

After deleting a source, run Data step 3 (Encode) again so from-scratch training sees the change. Rebuild the tokenizer (step 2) only before training a new model, never for a model you will resume (see section 5). Fine-tuning reads the folders directly. Models already trained keep what they learned.

# 12. Training: Cluster (several PCs train one model)

Several computers, at home or anywhere on the internet, can train **one** from-scratch model together. Every PC uses what it has: an NVIDIA GPU, an Intel Arc GPU, an Apple GPU or just the CPU. One PC is the **coordinator**: it holds the prepared data and the model. The others are **workers**.



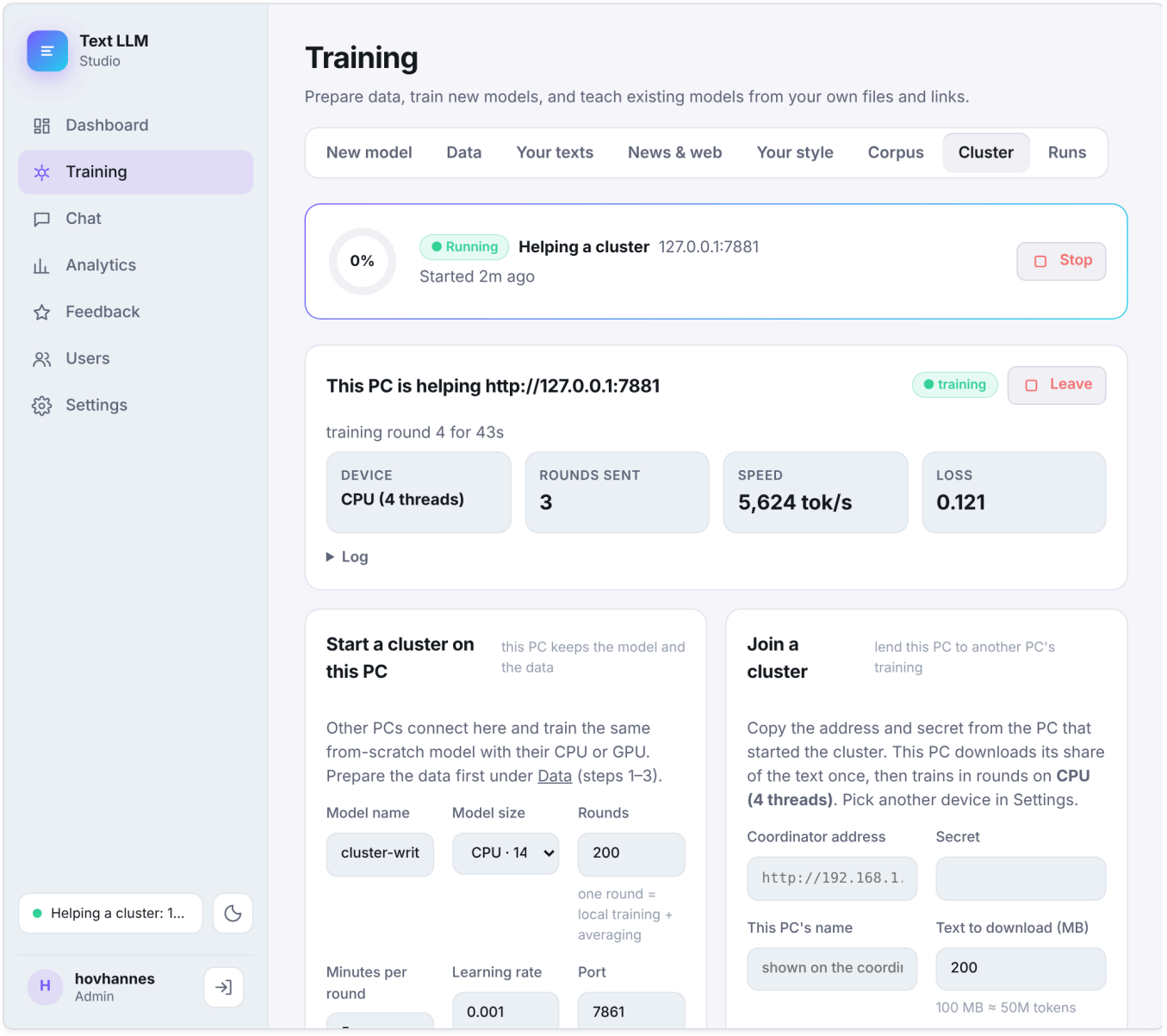

*A running cluster with three PCs*

**How it works.** Training goes in rounds (5 minutes by default). At the start of a round every PC holds the same model and trains on its own share of the text for the round's minutes. A GPU simply does more steps than a CPU, and nobody waits for the slowest PC. Then each PC sends how far its copy moved; the coordinator averages the moves (weighted by how much text each PC read) and takes one step with the average. Everyone continues from the result. The internet only carries about one model copy each way per PC per round (CPU size: 28 MB; Small: 220 MB), not data on every step, so a home connection is enough.

**Start a cluster** (on the PC with the data):

1. Run Data steps 1–3 first (section 5).
2. Training → **Cluster** → *Start a cluster on this PC*: name, model size, rounds, minutes per round, learning rate, port (7861). Leave **This PC trains too** on to use this PC as a worker as well, and **Let PCs on other networks join** on if any PC is somewhere else.
3. Press **Start the cluster**. The card at the top shows the addresses and the secret, plus a ready-made command for other PCs, with a **Copy** button. Pick the address the other PC can reach: **Internet** (any network), **Tailscale** (any network, both PCs on Tailscale) or **Same network**.

**Join from another PC:** either open its Studio → Training → Cluster → *Join a cluster*, paste the address and secret, and press **Join**; or, on a PC without the Studio, run `run.bat --join http://ADDRESS:7861 --secret SECRET` (`./run.sh` on Linux/macOS). The first run installs everything, including the GPU build of PyTorch when there is an NVIDIA or Intel Arc card. The worker downloads its share of the text once (200 MB by default, resumable), then trains. **Leave** (or Ctrl+C) takes it out; it can rejoin at any time.




*This PC helping a cluster*

**PCs on other networks.** Only the PC that starts the cluster has to be reachable. The workers only connect out to it, like a browser opening a website, so a worker behind any home router or on mobile internet needs no setup.

- With **Let PCs on other networks join** on, the coordinator asks its router to open the port (UPnP, switched on in most home routers) and closes it again when the cluster stops. When that works the card says ✔ **Reachable from other networks** and lists an **Internet** address with your public IP.
- When it says **Same network only**, the router refused or is not the one facing the internet (UPnP switched off, a second router behind the provider's box, or a provider that shares one public address between customers, which is common on mobile internet). Then install the free **Tailscale** on every PC and use the **Tailscale** address, or forward TCP port 7861 to this PC in the router's settings and use your public IP.

The long secret keeps strangers out, but the traffic itself is not encrypted, so Tailscale is the most private choice.

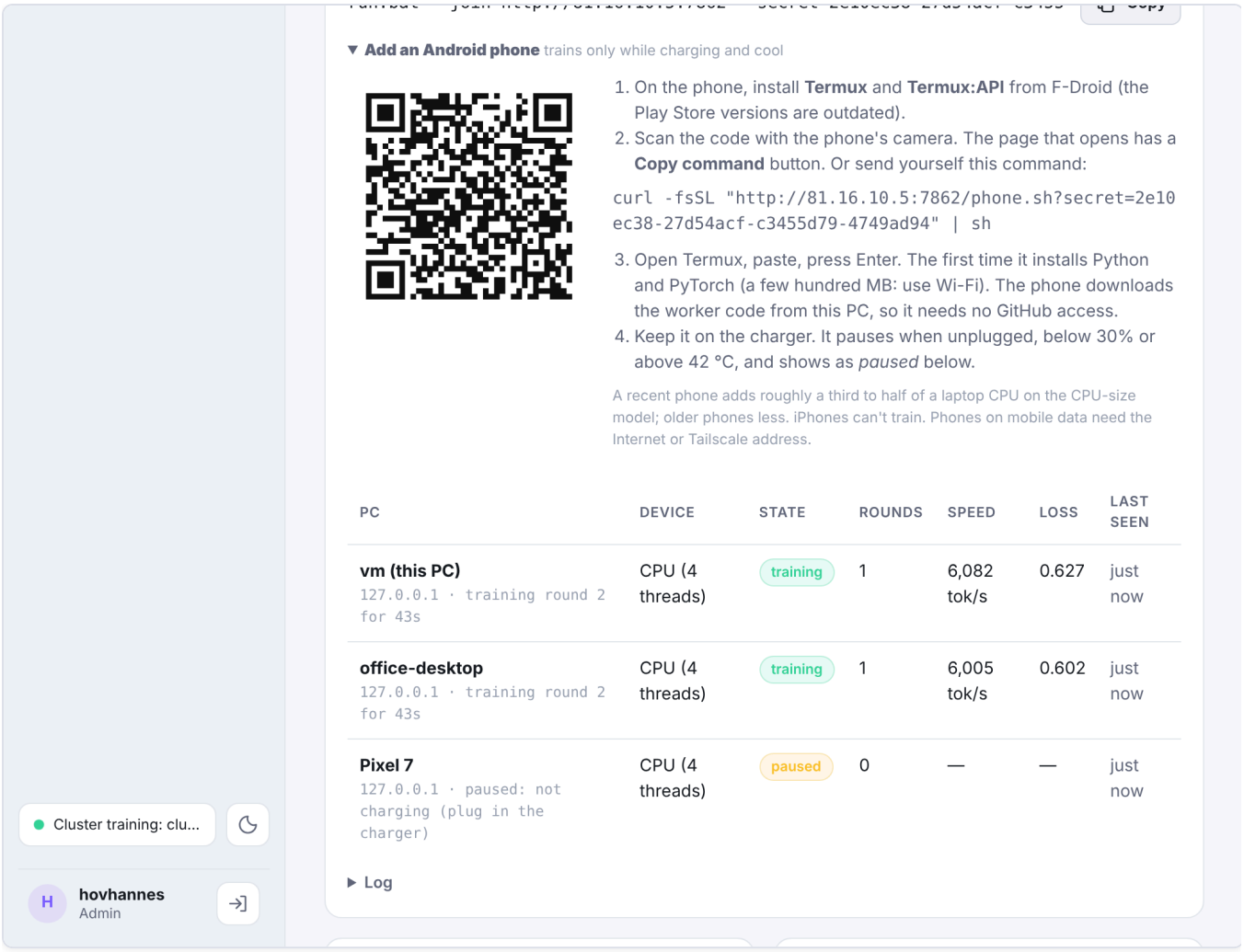
**What speedup to expect.** Roughly 0.6–0.9× each extra PC's own speed: four similar PCs train about 3× faster than one. A GPU worker adds much more than a CPU one. Rounds of 5–10 minutes keep the traffic low; shorter rounds help quality a little but cost more upload.

**Advanced:** secret (made up for you), workers needed before the first round, final learning rate, warmup, weight decay, outer learning rate (0.7) and momentum (0.9), how often to check quality, this PC's batch size, and **Resume** (continue a stopped cluster model from its last round; workers reconnect by themselves). Workers' advanced options: batch size, gradient accumulation, "stop early" seconds for a slow upload, and using the PC's own prepared data instead of downloading (only with the same tokenizer).

**Kind of model:** *Writer* uses the Data tab's prepared text; *Copilot (code)* uses the code prepared on the Copilot tab (section 10). A worker that uses its own prepared data picks the same kind under *Own data is for*.

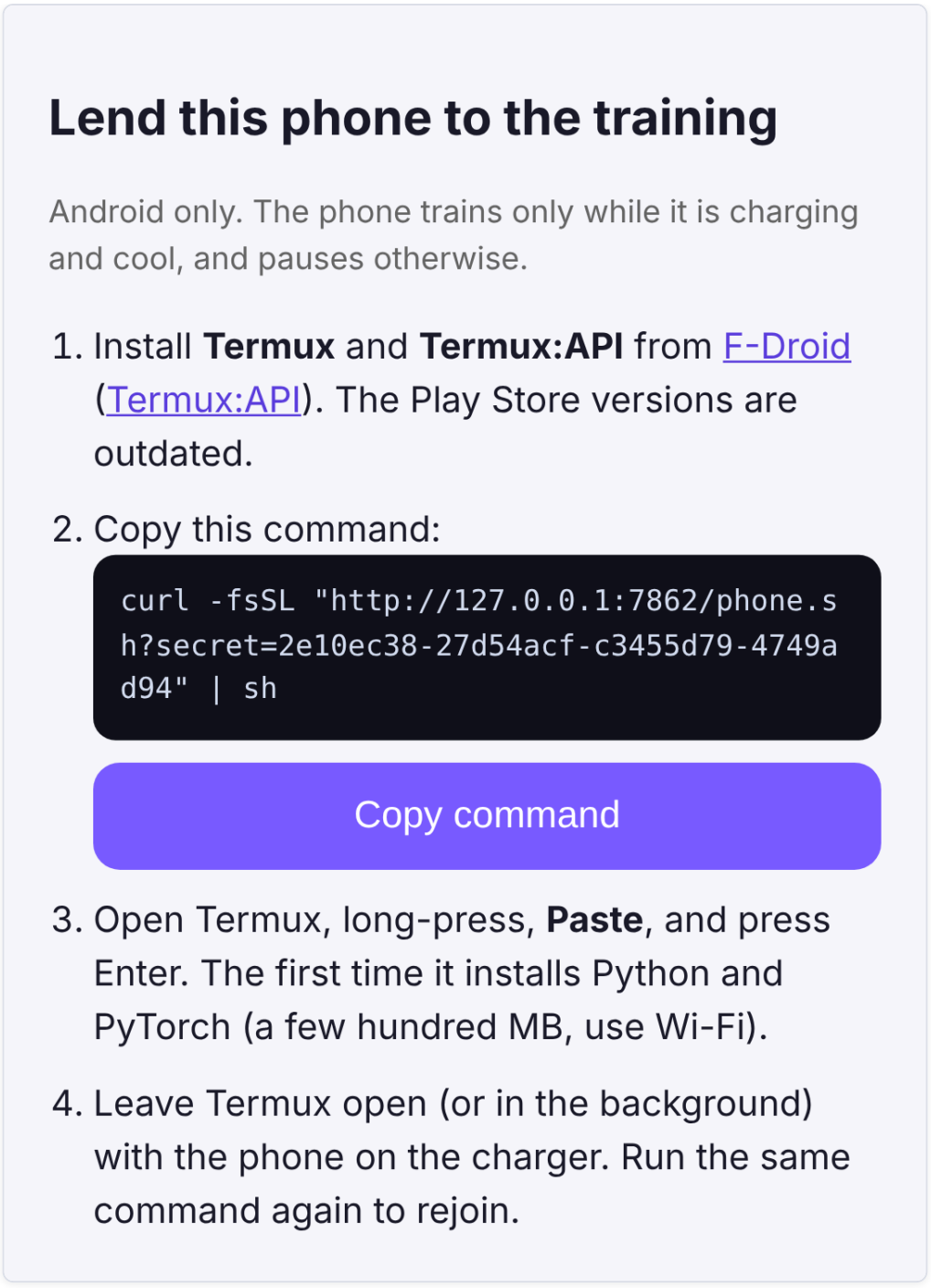
## Android phones

Android phones can join too, through the free **Termux** app. A phone trains only while it is **charging**, above 30% and below 42 °C; otherwise it shows as *paused* in the table and the others carry on without waiting for it. iPhones can't run training.




*Adding a phone: QR code and command*

1. On the phone, install **Termux** and **Termux:API** from F-Droid (the Play Store versions are outdated; Termux:API lets the worker read the battery).
2. While the cluster runs, open **Add an Android phone** on the Cluster card and pick the address the phone can reach (on mobile data: Internet or Tailscale). Scan the QR code with the phone's camera.
3. The page that opens has a **Copy command** button. Open Termux, paste and press Enter. The first time it installs Python and PyTorch (a few hundred MB, use Wi-Fi). It downloads the worker code from your PC, so the phone needs no GitHub access and always runs the same version.
4. Leave the phone on the charger with Termux open or in the background. Run the same command again to rejoin later.



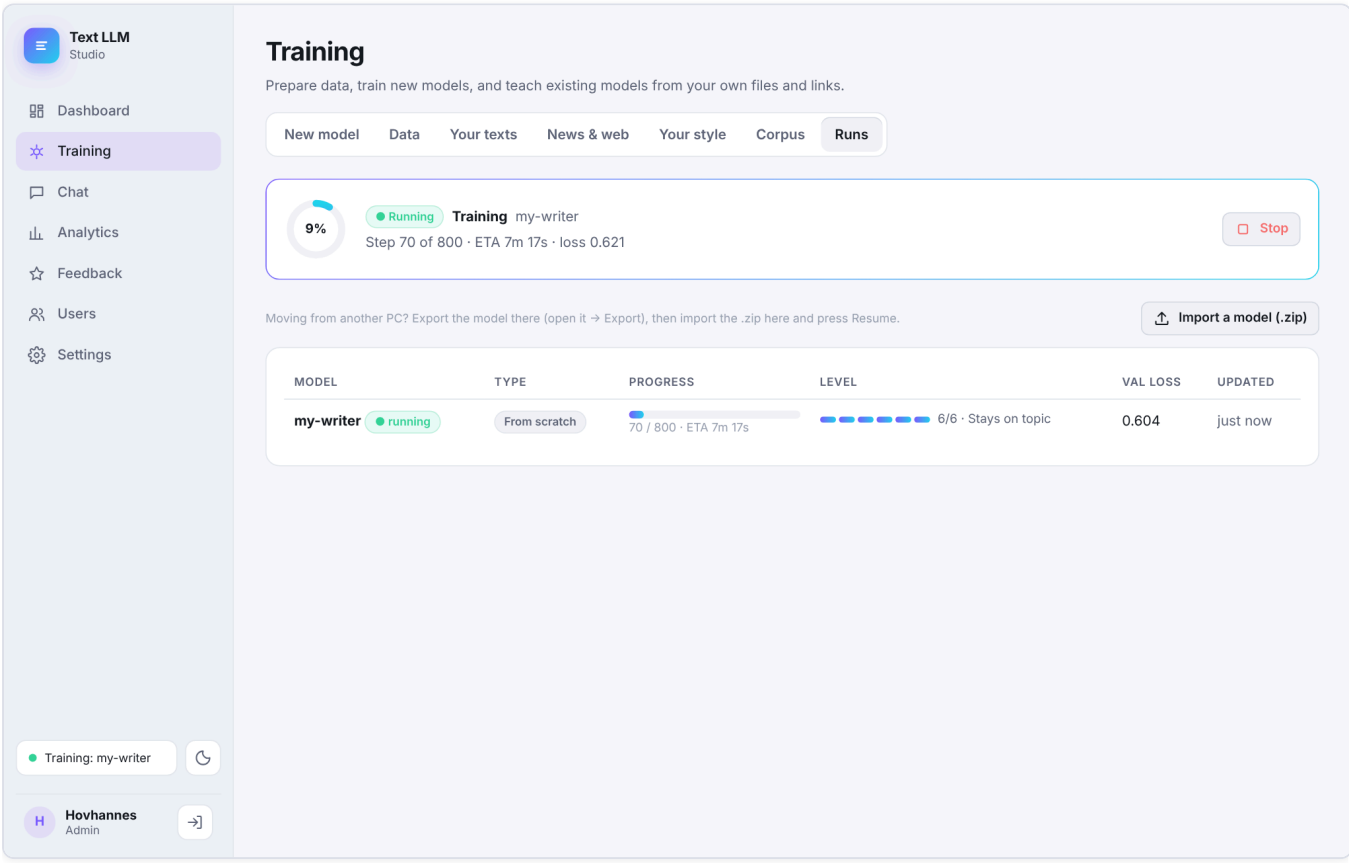

*The page the QR code opens on the phone*

**What a phone adds:** a recent phone is roughly a third to half of a laptop CPU on the CPU-size model, an older one much less, and phones slow down when warm. The Small size and up are too heavy for phones. Think of phones as a bonus overnight on the charger, not a replacement for PCs.

The result is an ordinary from-scratch run under Runs: chat with it, export it, or keep training it alone with Resume. Cluster mode is for from-scratch models; fine-tuning (LoRA) still runs on one PC.

# 13. Training: Runs

Every model you train is a **run** in `runs/<name>/`.

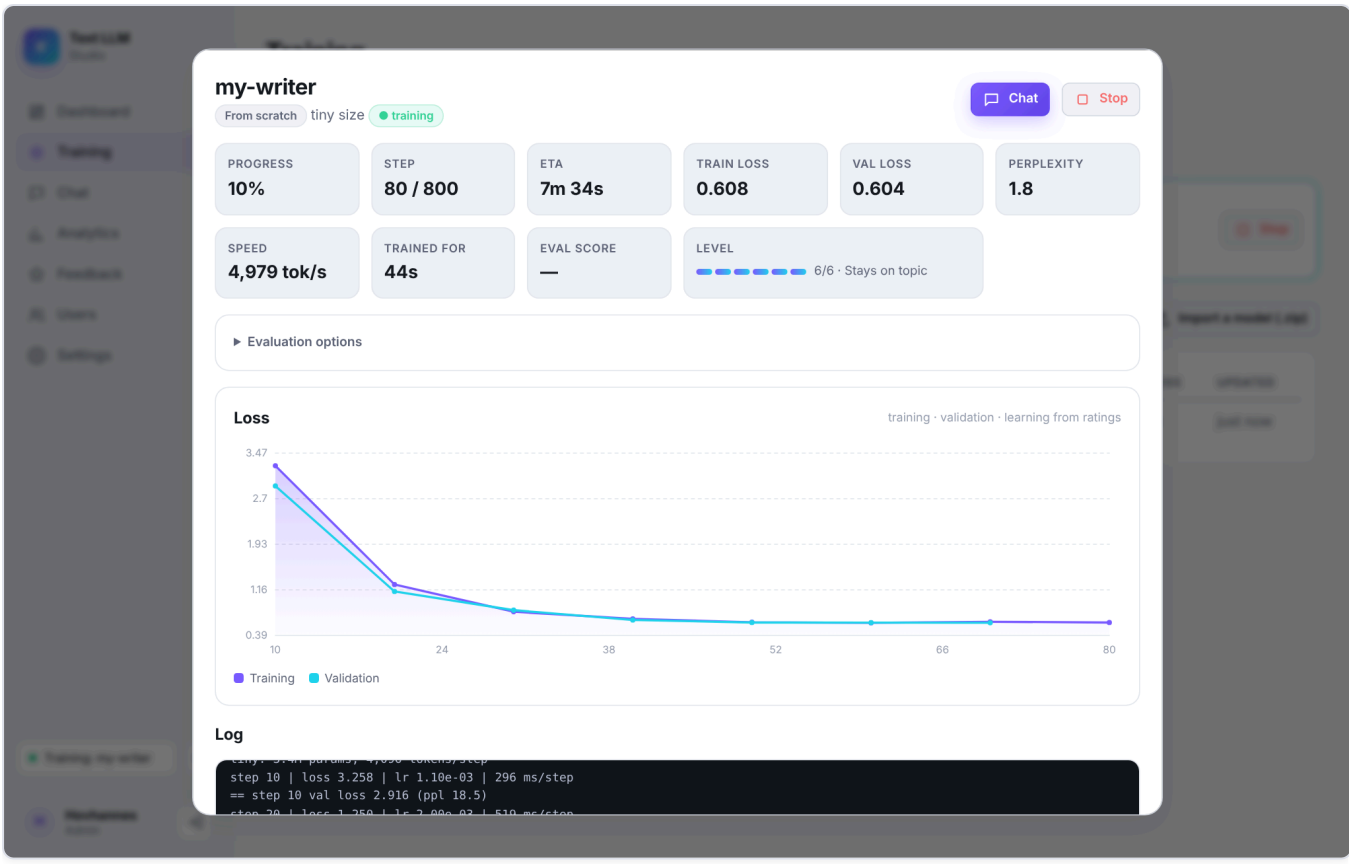



*Runs while a model is training*

The table shows each run's type, progress bar with step count and ETA, level and validation loss. A running job has a green "● running" tag, and the live card at the top has a **Stop** button. Stopping keeps the last checkpoint.

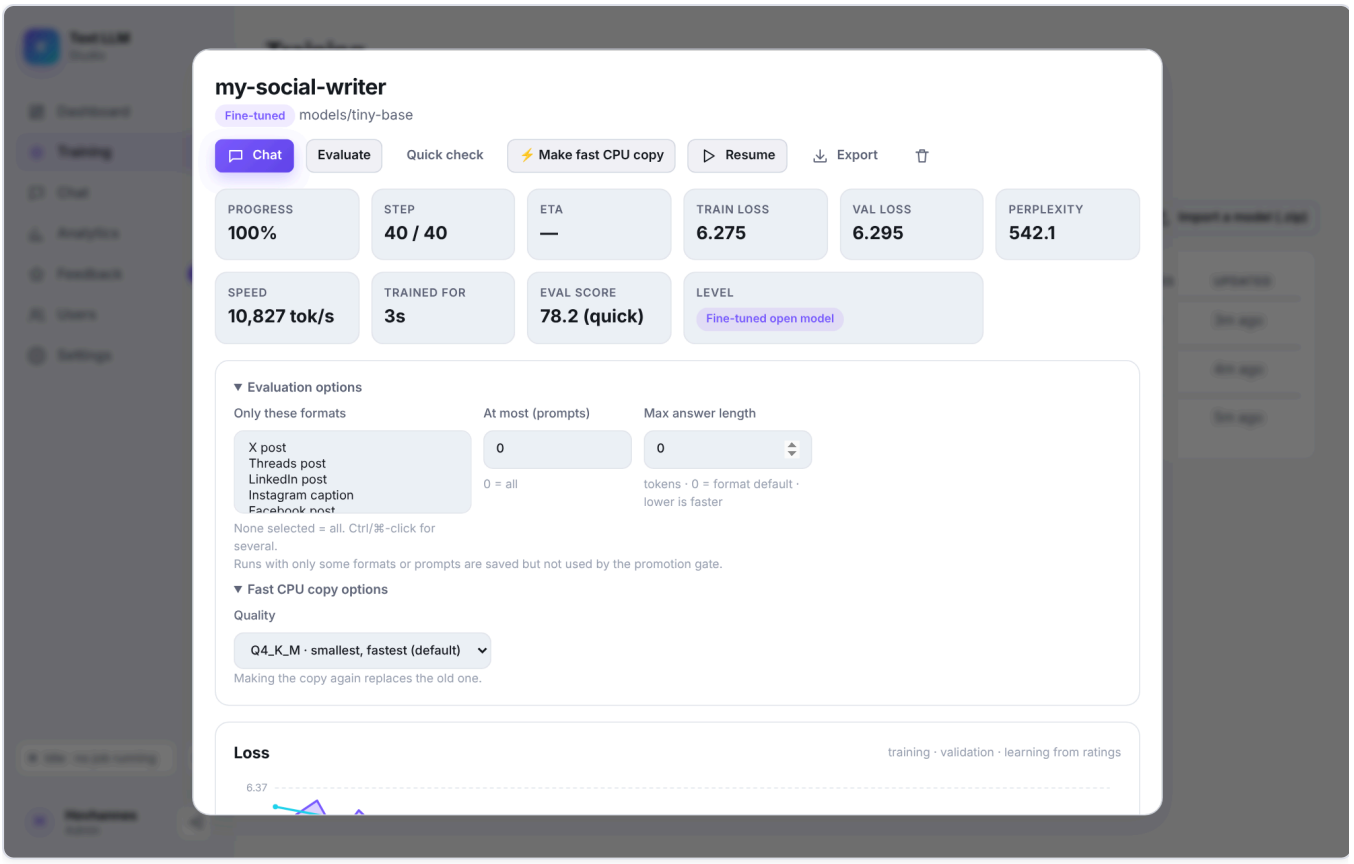
## Run details

Click a run to open its details. They refresh every few seconds.




*Details of a from-scratch run while training*

- **Stats:** progress, step, ETA, train and validation loss, perplexity, speed (tokens per second), time trained, evaluation score, and level.
- **Loss chart:** training, validation and learning-from-ratings curves.
- **Log:** the full output of the training script.




*Details of a fine-tuned run with all buttons*

Buttons (which ones appear depends on the run's state):

| Button | What it does |
|---|---|
| **Chat** | open Chat with this model |
| **Evaluate** | score it on the 60 fixed prompts (all formats) |
| **Quick check** | 8 prompts, a few minutes |
| **Make fast CPU copy** | (fine-tuned only) build a 4-bit GGUF file; chat then uses it |
| **Stop** | stop the running job; the last checkpoint is kept |
| **Resume** | continue training from the last checkpoint with the run's own settings |
| **Export** | download the run as a `.zip` (from-scratch: OK = include the training data) |
| **Retrain on new data** | open the Retrain tab with this model chosen: a new model that continues it on other data (section 7) |
| 🗑 | delete the run and its weights |

**Evaluation options:** only some formats, at most N prompts, maximum answer length. Partial evaluations are saved but not used when deciding the default model. **Fast CPU copy options:** quality Q4_K_M (smallest, fastest, default), Q5_K_M, Q6_K or Q8_0 (bigger, closer to the original).

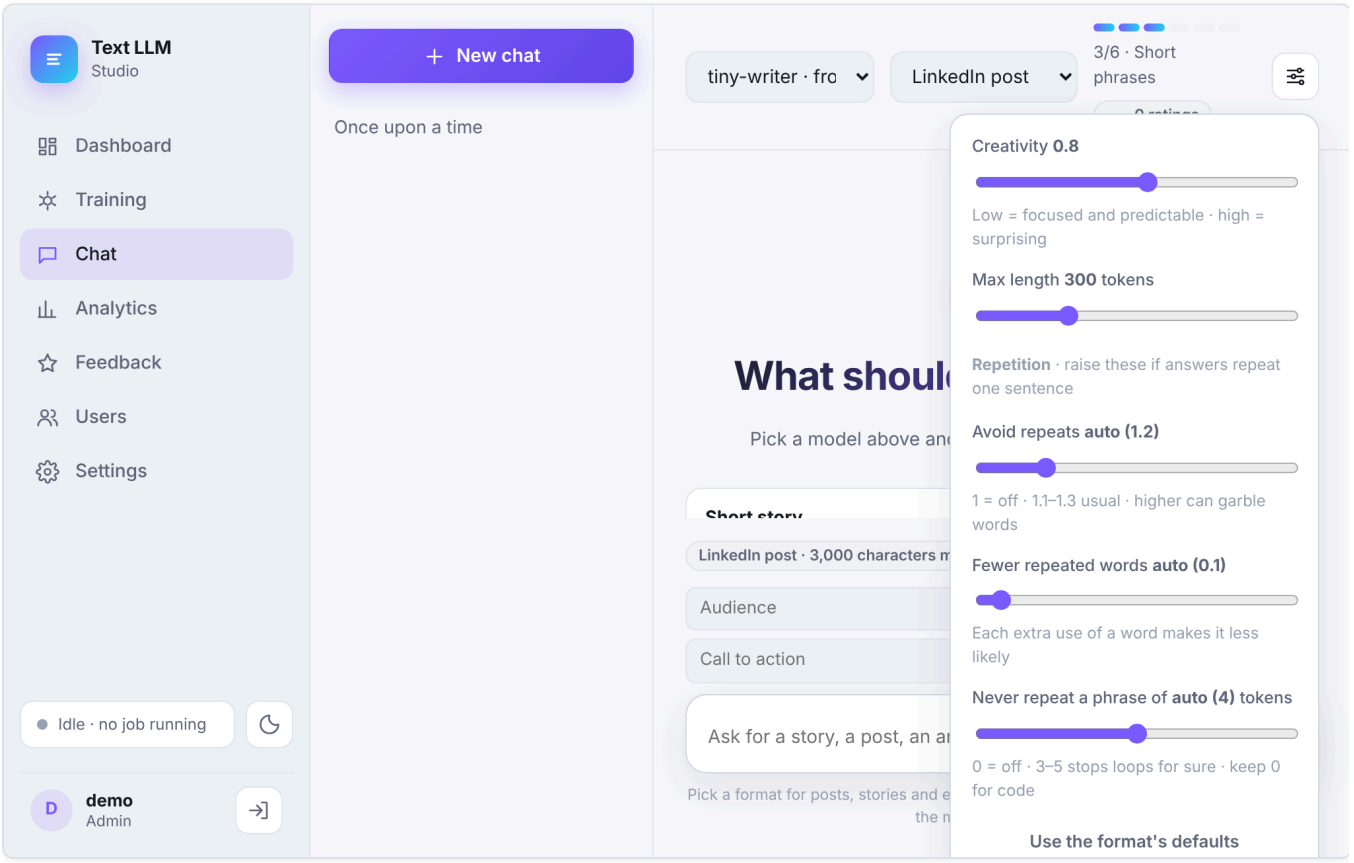
## Move a model to another PC

1. On the old PC open the model, press **Stop** if it is training, then **Export**.
2. On the new PC run `run.bat` once, go to Training → Runs and press **Import a model (.zip)** at the top right. If a model with that name exists you can give a new name or replace it.
3. Open the imported model and press **Resume**.

This works between Windows and Linux, and between GPU and CPU computers. A fine-tuned model's base model (for example Qwen3 0.6B) is not inside the zip; the new PC downloads it once.

# 14. Chat

Chat is where you (and regular users) write with the models.



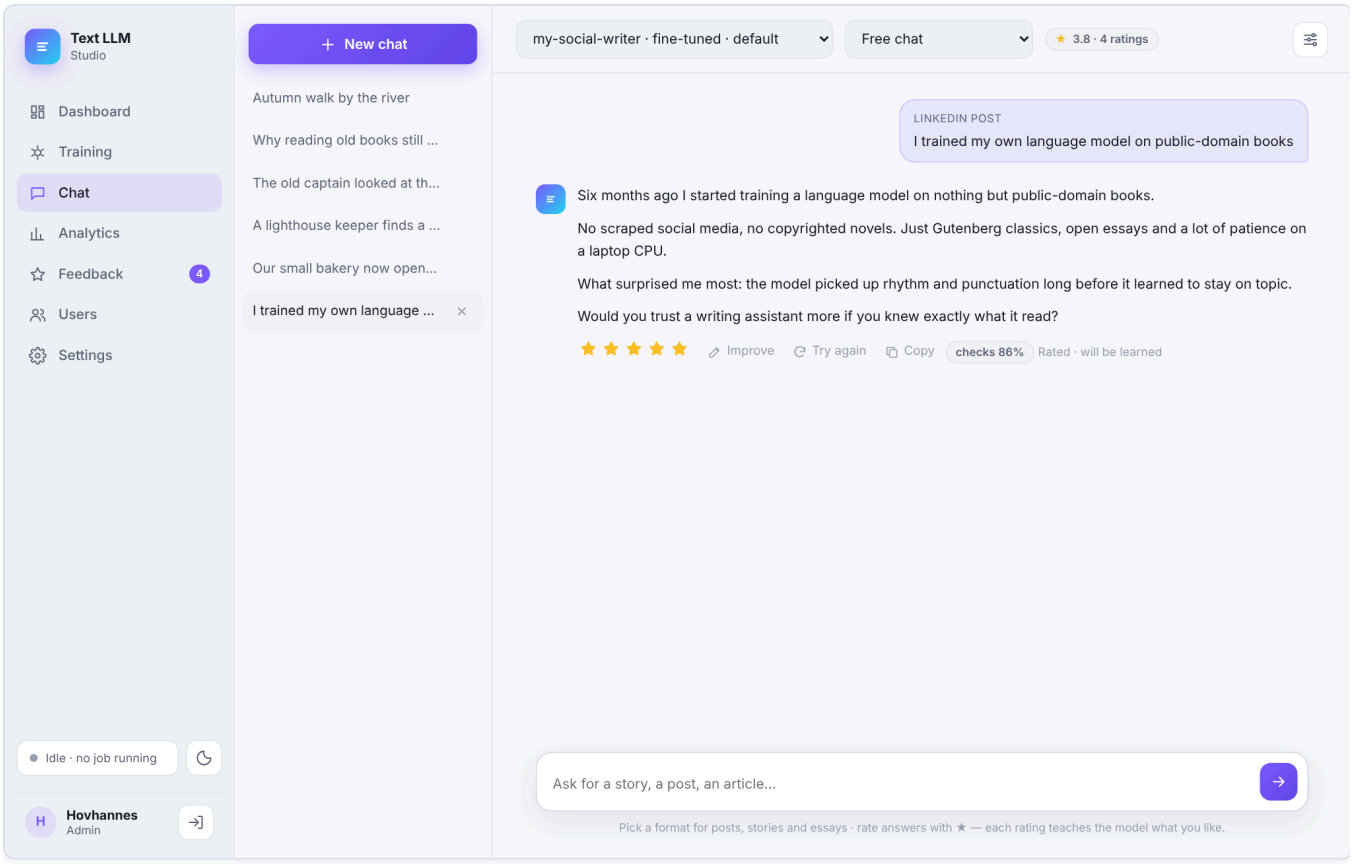

*Chat: a new conversation with the LinkedIn post format and the settings popover*

- **Left:** your conversations. **New chat** starts a fresh one; the ✕ deletes one.
- **Model picker:** every trained model with its type (“from scratch”, “fine-tuned”, “writes like …”) and which one is the **default**. Next to it you see the model’s level and average rating.
- **Format picker:** Free chat; posts (X post, Threads post, LinkedIn post, Instagram caption, Facebook post); About today’s news; Short story; Essay; Write like a style profile. Each format has its own instructions, length limit and a small form (tone, audience, takeaway, hashtags, length…) shown above the message box.
- **Settings button** (sliders icon): **Creativity** (low = focused, high = surprising) and **Max length** in tokens, plus three **Repetition** controls that stop a model from writing the same sentence over and over:
  - **Avoid repeats** (repetition penalty): makes every word already in the text less likely. 1 = off, 1.1–1.3 is usual; much higher starts to garble words.
  - **Fewer repeated words** (frequency penalty): each extra use of a word costs a little more. 0 = off.
  - **Never repeat a phrase of N tokens** (no-repeat n-gram): the model may never write the same run of N tokens twice, which ends loops for sure. 0 = off; 3–5 for stories and posts; keep 0 for code.

Each control shows **auto (value)** until you move it. Auto is firmer for models trained from scratch (1.2 / 0.1 / 4) than for fine-tuned ones (light repetition penalty only), and off for the code formats. *Use the format's defaults* resets all of them.

- The welcome screen has example prompts; click one to try it.

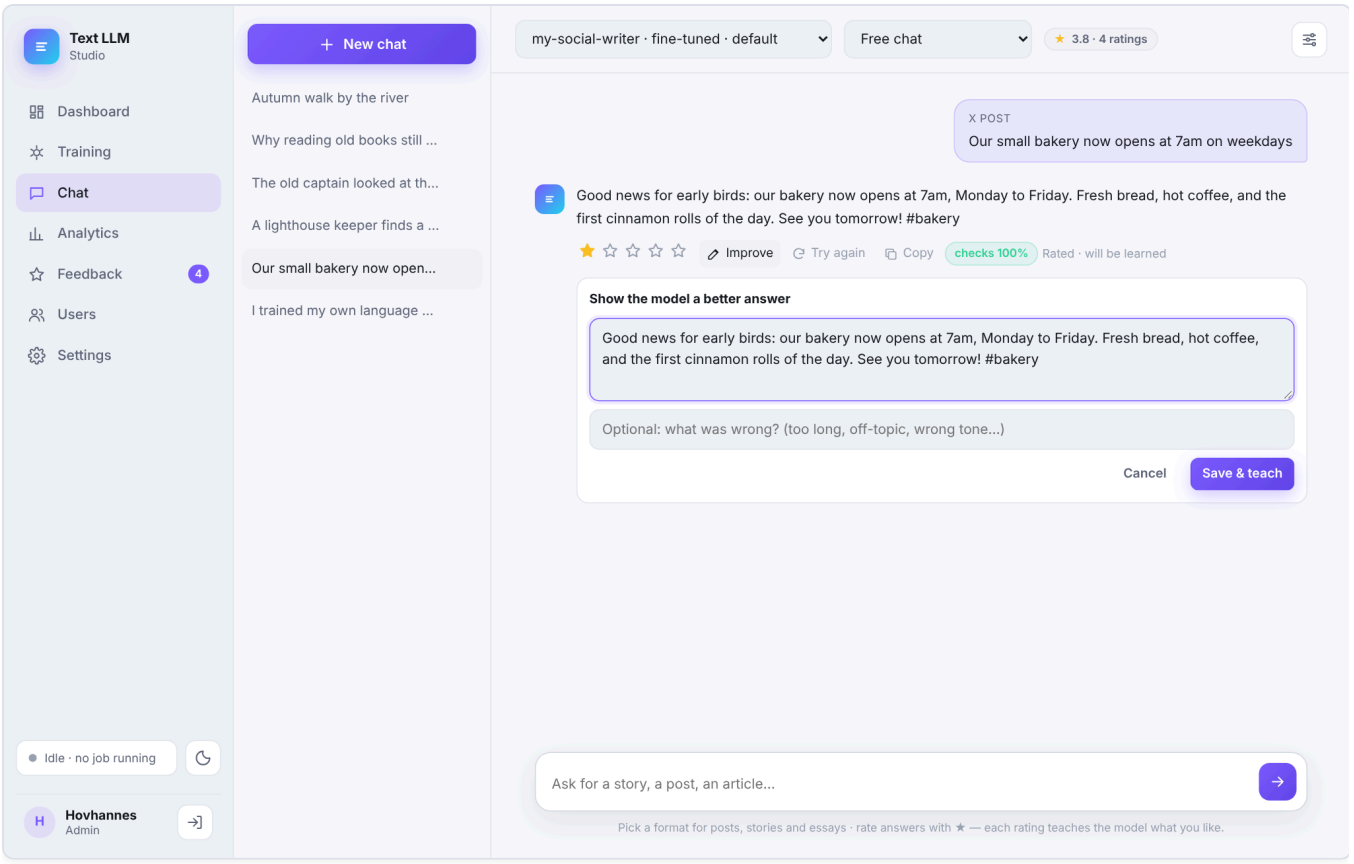
Press **Enter** to send, **Shift+Enter** for a new line. The answer streams in. Stories and essays are planned first; the plan appears in a **Plan** box above the answer. Posts are checked automatically (length limit, hashtags, hook, closing question) and rewritten up to 3 times if they fail.




*An answer with its rating, actions and format-check score*

Under every answer:

- **★ 1–5:** rate it. Each rating teaches the model what you like. A rating of 1–2 opens the *Improve* box.
- **Improve:** write or paste the answer you wanted, optionally say what was wrong, and press **Save & teach**. The model learns from your corrected answer.
- **Try again:** writes another answer to the same request ("Answer 2 of 2").
- **Copy:** copies the text.
- **checks N%:** the automatic format checks (green = all passed).
- A status: "Rated · will be learned", "✓ learned", or "held back by the quality gate".




*Improve: show the model a better answer*

## How much training is enough

A model trained from scratch only **continues text**: it has never seen an instruction. So Chat turns your message into the start of a document for it: with **Short story**, “A lighthouse keeper finds a message in a bottle” becomes a title, a blank line and “Once upon a time,” and the answer starts with those opening words (the *setting* and *characters* fields go into the title). Essays and posts get the title only. In **Free chat** a request such as “Write a little story about a fox” is treated the same way; anything else is taken as an opening to continue (“The old fox looked at the river and”). The model can still wander off topic: it writes the kind of text it read most. How well it continues depends on its **level** (Dashboard and Runs, from the validation loss):

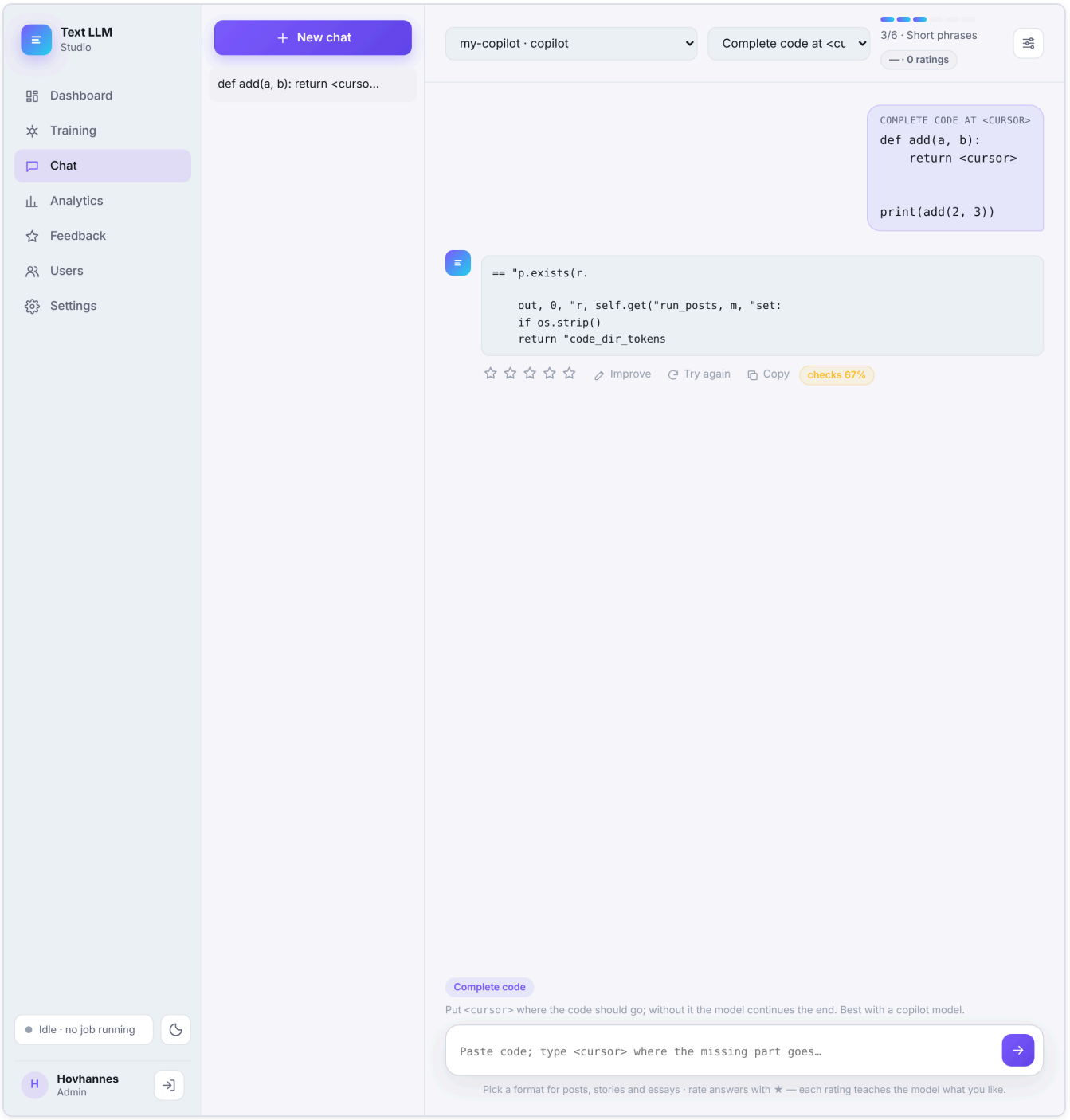
| Level (val loss) | What chat looks like |
|---|---|
| 1–3 (above 4.0) | Words and short phrases; without the Repetition controls it often repeats one sentence. Keep training. |
| 4 Grammatical sentences (3.4–4.0) | Correct sentences that drift from one topic to the next. |
| 5 Fluent paragraphs (3.0–3.4) | Readable paragraphs; a story keeps its characters for a few sentences. |
| 6 Stays on topic (below 3.0) | Short stories that mostly hold together. |

Rough budgets: the **cpu** size (~14M weights) with the README’s settings (16 × 4 × 256 ≈ 16k tokens per step) needs about **20,000 steps** (≈330M tokens, a few days on a laptop CPU) to reach level 4–5, and it stays a “book-flavoured continuer” at best. Coherent stories from scratch need the **small** size

(~110M) trained on a few billion tokens on a GPU, ideally with simple story text mixed in (*More book sources → Cosmopedia, subset stories*). The data decides the voice: a corpus that is mostly FineWeb-Edu (web pages) gives web-page answers ("You can use this to learn more..."); for stories, give books and stories the biggest share (Gutenberg, PG-19, Standard Ebooks, Cosmopedia stories) and keep FineWeb small. For stories, posts and following instructions on a normal PC, the practical route is a **fine-tuned open model** (section 4.2, e.g. Qwen3-0.6B) with the **Short story** format.

With **About today's news**, the answer lists the articles it is based on, with links.

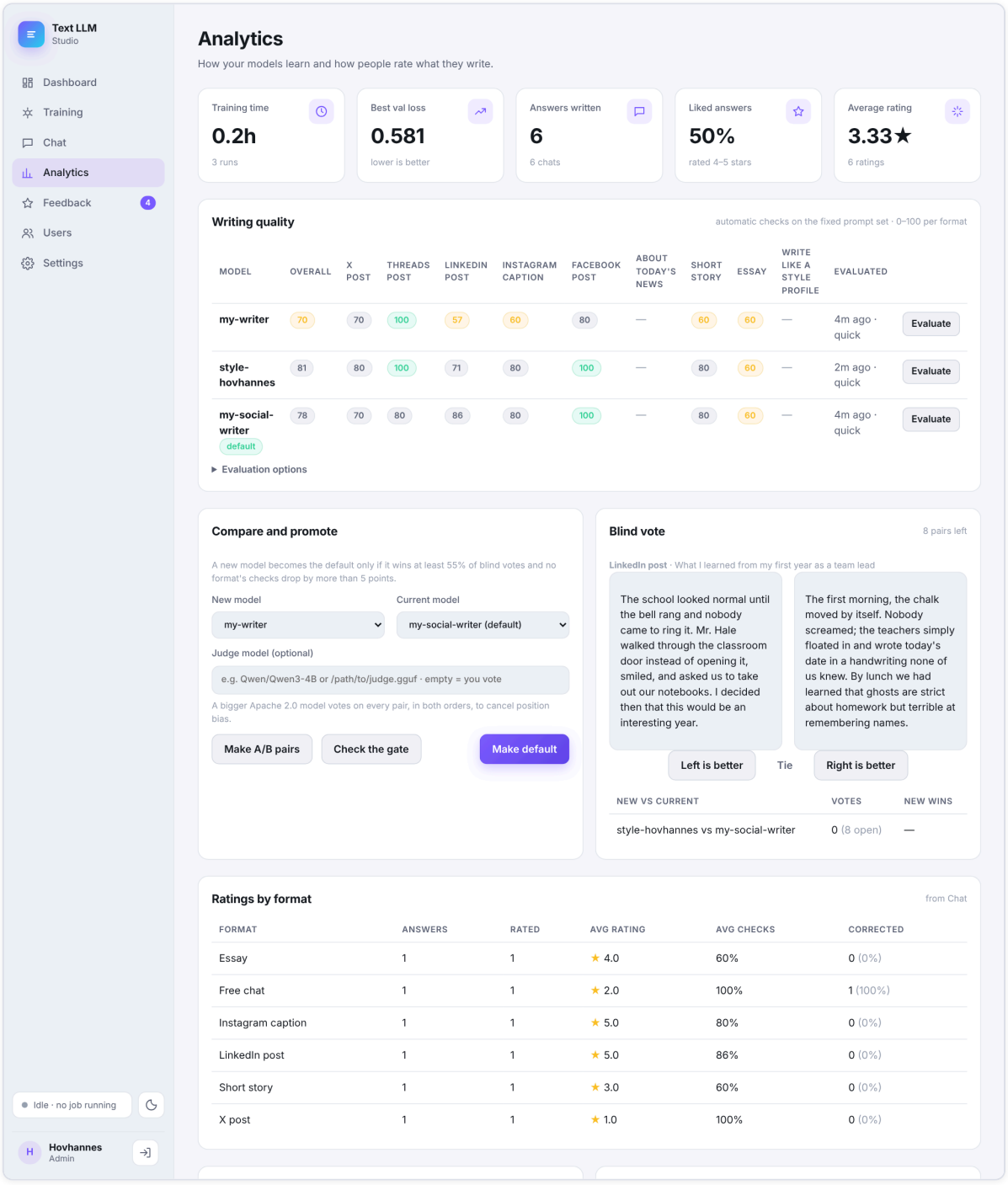
**Code formats** (for copilots, section 10): **Code assistant** answers programming requests with code blocks; **Complete code at <cursor>** fills in the spot marked `<cursor>` in the code you paste. Choosing a copilot model switches to a code format by itself. Code answers are shown in a code box and checked for balanced brackets and repeated lines.




*Complete code with a (tiny demo) copilot*

# 15. Analytics

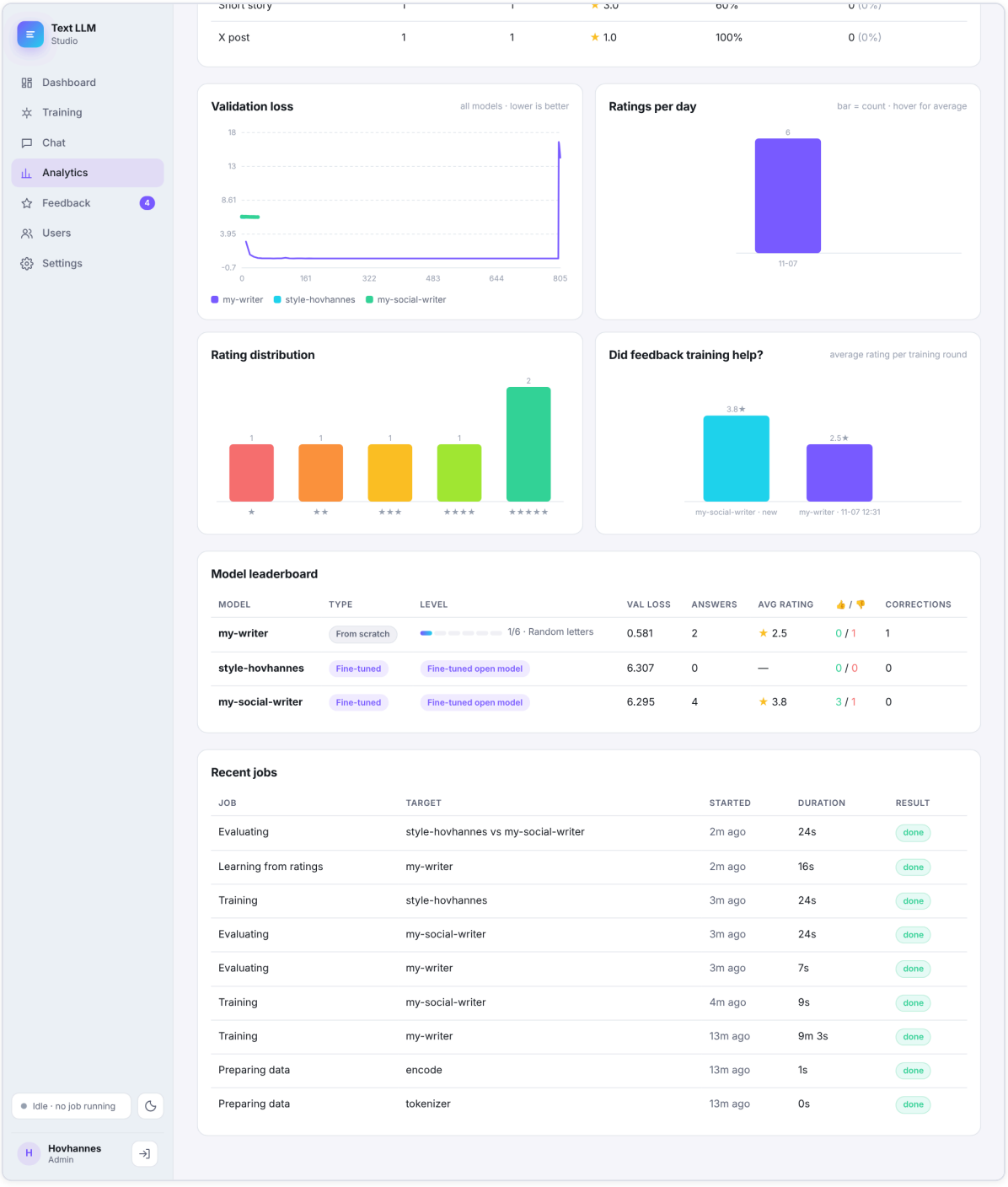
Admins see how the models learn and how people rate them.




*Analytics: KPIs, writing quality, compare and promote, blind vote*

- **KPIs:** total training time, best validation loss, answers written, share of liked answers (4–5★), average rating.
- **Writing quality:** the automatic-check score (0–100) of every model overall and per format, from its latest evaluation, with **Evaluate** and **Quick** buttons. The default model is marked *default*; ⚡ *fast* means chat uses the GGUF copy.

- **Compare and promote:** pick a **new model** and the **current model**. **Make A/B pairs** has both write answers to the same prompts. Optionally name a bigger **judge model** that votes on every pair. **Check the gate** tells you whether the new model may become the default, and **Make default** promotes it. A model becomes the default only if it wins at least 55% of at least 10 blind votes and no format's checks drop by more than 5 points.
- **Blind vote:** two answers to the same prompt, without model names. Choose **Left is better**, **Tie** or **Right is better**. The table below counts votes and wins.

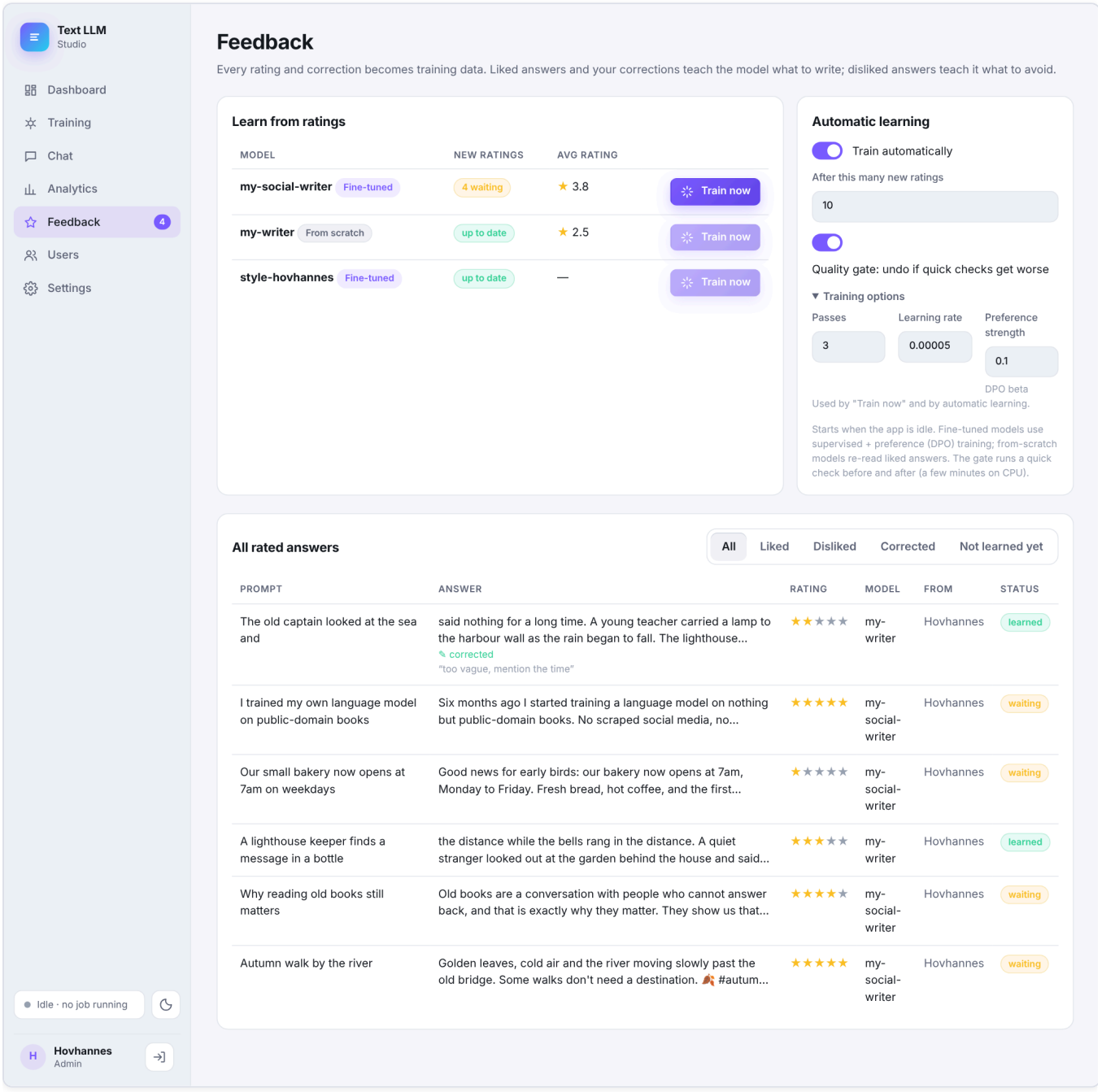



*Analytics: ratings by format, charts, leaderboard and jobs*

Further down: **ratings by format**, **validation loss** for all models, **ratings per day**, **rating distribution**, **Did feedback training help?** (average rating per training round), the **model leaderboard** (click a model to open its details) and **recent jobs** with their result.

# 16. Feedback

Every rating and correction becomes training data. Liked answers and corrections teach the model what to write; disliked answers teach it what to avoid.




*Feedback*

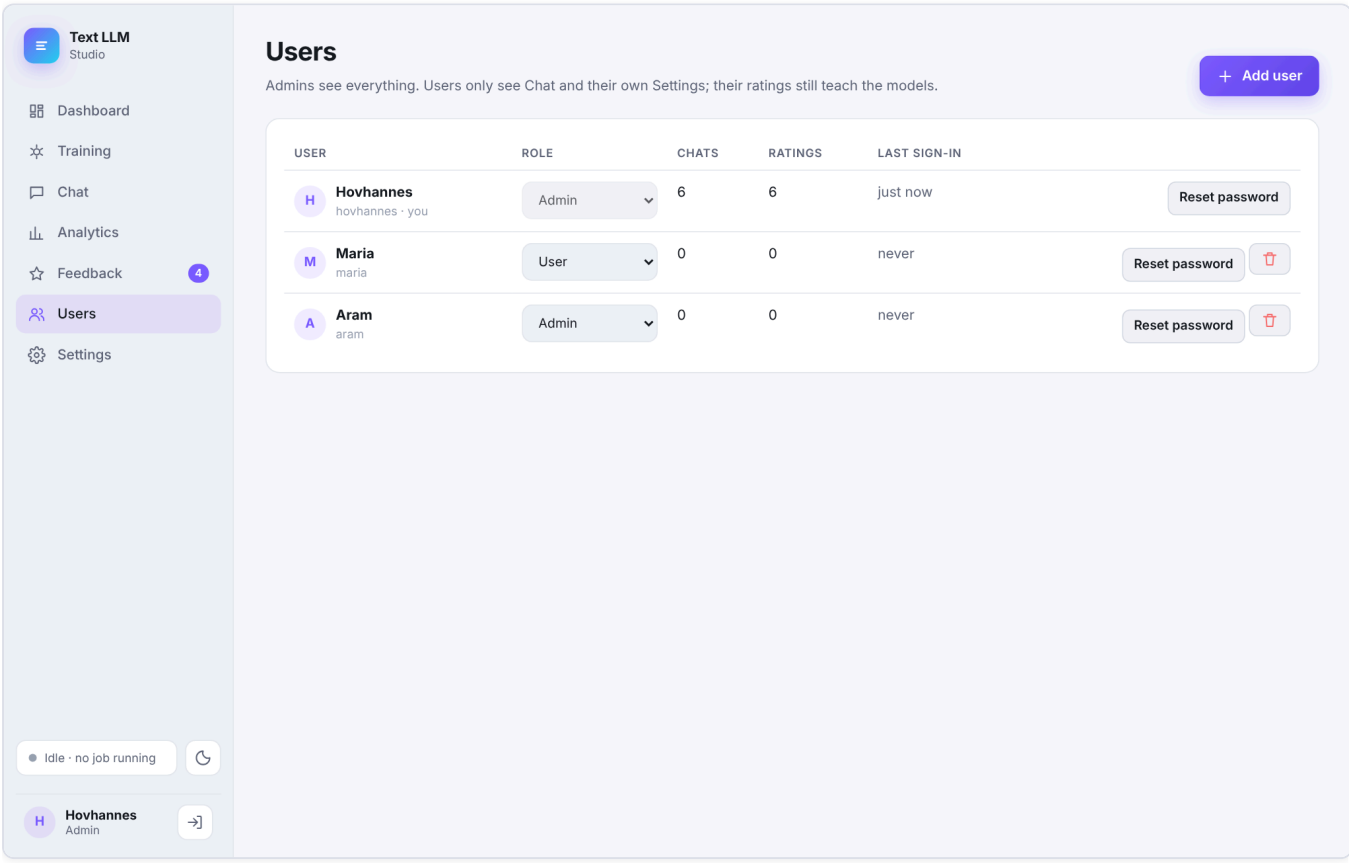
- **Learn from ratings:** one row per model with the number of ratings waiting, its average rating and **Train now**.
- **Automatic learning:** *Train automatically* after N new ratings (default 10), and the **quality gate**: a quick check before and after, and the learning is undone if the checks get worse.
- **Training options:** passes, learning rate and preference strength (DPO beta). Used by *Train now* and by automatic learning.

- **All rated answers:** filter by All, Liked, Disliked, Corrected or Not learned yet. Each row shows the prompt, the answer (✎ corrected, your comment), rating, model and status: *waiting*, *learned* or *held back*. Click a row to open that conversation.

How the learning works: fine-tuned models train on liked and corrected answers directly and use a DPO preference loss when a liked or corrected answer can be paired with a disliked one. From-scratch models re-read the prompts with their liked answers. Feedback that was used is marked *learned* and is not trained on again.

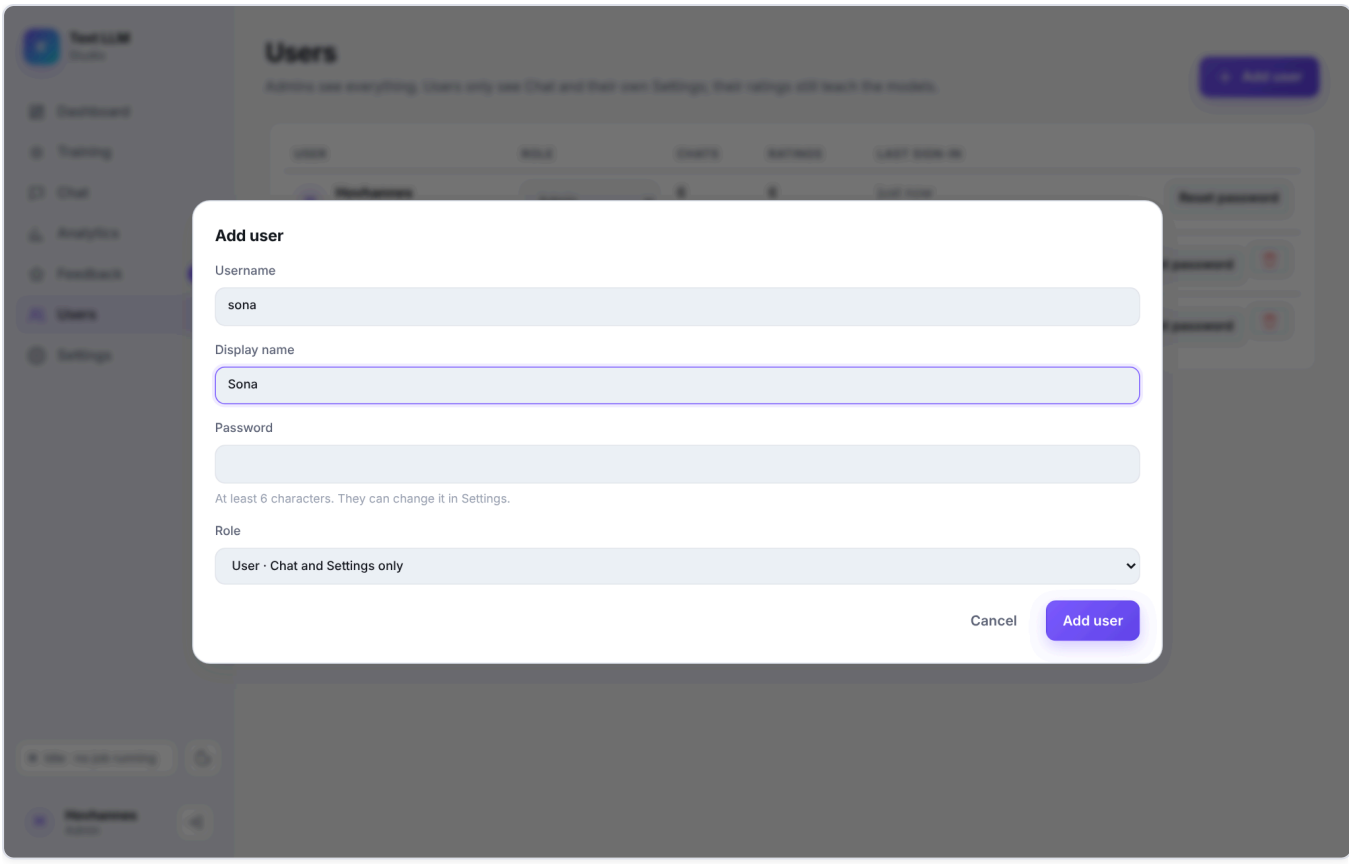
# 17. Users

Admins add people and set their role.




*Users*

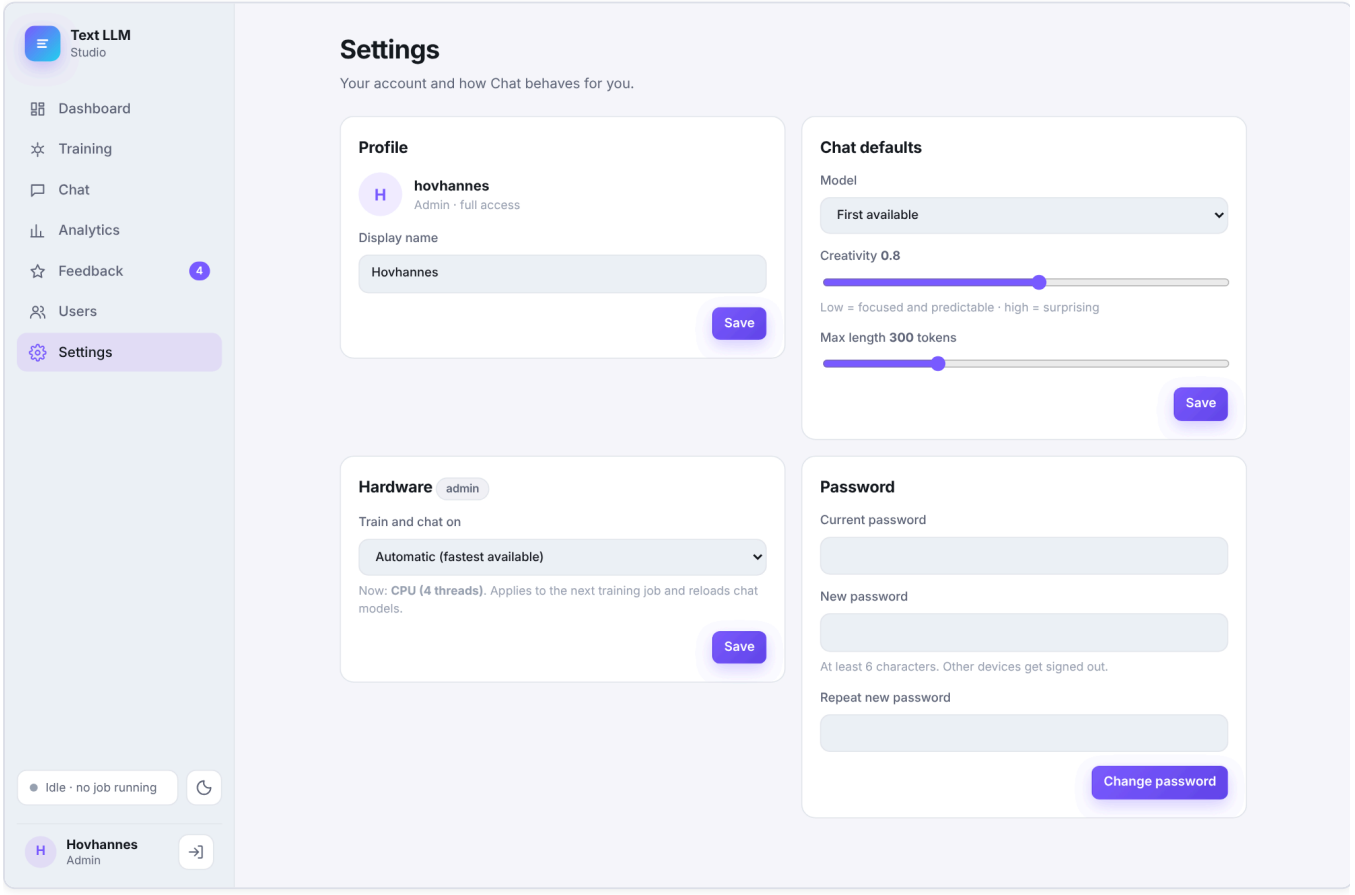
Each row shows the person, their role (change it in the drop-down; you can't change your own), how many chats and ratings they have, their last sign-in and **Reset password**. The trash button removes an account (you cannot remove yourself).




*Add user*

**Add user** asks for a username, an optional display name, a password (at least 6 characters; they can change it later) and the role: **User** (Chat and Settings only) or **Admin** (everything).

# 18. Settings and hardware




*Settings (admin view with Hardware)*

- **Profile:** your display name.
- **Chat defaults:** the model Chat opens with, creativity and maximum length.
- **Password:** current password and a new one twice. Other devices get signed out.
- **Hardware** (admins only): **Train and chat on** automatic, CPU, or a specific GPU. It applies to the next training job and reloads chat models. Intel Arc cards appear as `xpu` (and `xpu:1` for a second card). If you have an NVIDIA card but only the CPU build of PyTorch is installed, the hint tells you to close the Studio and run `run.bat --gpu`; for an Intel Arc card it says `run.bat --xpu`. On an Arc card the hint also lists what runs on the CPU instead.
- **Links** (admins only): **Allow links to the local network**. Off by default: links you give in Your texts, news sources and style imports are only read from public internet addresses, never from this PC, the router or other PCs on your network (this matters when the Studio is reachable from the internet). Turn it on to read from a NAS or an intranet site.
- **Fast engines** (admins only): which compiled libraries are installed (llama.cpp for fast GGUF chat, the Rust tokenizer, faiss, ctranslate2, onnxruntime) and the pip command for each missing one. Everything works without them; they make parts faster.

**Safety when the Studio faces the internet.** Every `/api` call needs a signed-in account; eight wrong passwords in a row lock that username for a minute; cross-site requests are refused; model files are only ever loaded as plain tensors (never as code), so an imported model bundle from someone else

can’t run anything on your PC; and a bundle’s saved training command is checked before Resume uses it.

# 19. Command-line equivalents

Everything in this manual can also be done from a terminal in the project folder (activate `.venv` first). The most useful commands:

```
python -m studio.auth add alice --admin     # add an admin (asks for a password)
python -m studio.auth add bob               # add a regular user
python -m studio.auth passwd bob            # new password
python data/build_corpus.py --gutenberg-max 3000 --fineweb-gb 0.3
python tokenizer/train_tokenizer.py --raw data/raw --vocab 8192 --out tokenizer/bpe.json
[--style modern|gpt2] [--lang-balance 0.3] [--sample-mb 4]
python tokenizer/train_tokenizer.py --check --tok tokenizer/bpe.json --sample-mb 8   #
quality numbers
python tokenizer/train_tokenizer.py --encode --raw data/raw --tok tokenizer/bpe.json --bin
data/bin
python train.py --preset cpu --vocab 8192 --max-steps 20000 [--resume]
python train.py --preset cpu --vocab 8192 --procs 2 --threads 0   # 2 processes on this PC
(0 = auto)
python -m core.parallel --preset cpu --batch 16                  # speed test: which --
procs is fastest here
python finetune/finetune_lora.py --base Qwen/Qwen3-0.6B --out runs/my-social-writer --gguf
python learn.py --run runs/my-writer --files book.pdf --urls https://example.com/article
python retrain.py --from runs/my-writer --out runs/my-writer-v2 --source
data/raw/news,data/raw/custom --steps 2000
python retrain.py --resume --out runs/my-writer-v2   # continue a stopped retrain
python data/crawl.py --learn runs/my-model
python data/own_posts.py --profile me --name "Me" --x-archive twitter.zip --i-own-these
python generate.py --ckpt runs/my-model/ckpt.pt --tok runs/my-model/tokenizer.json --
prompt "Once upon a time" --repetition-penalty 1.2 --no-repeat-ngram 4
python evaluate.py --run runs/my-social-writer --quick
python transfer.py export --run runs/my-model --with-data
python transfer.py import my-model.textllm.zip
python cluster.py coordinator --out runs/cluster-writer --preset cpu --vocab 8192 --local-
worker
python cluster.py worker --join http://ADDRESS:7861 --secret SECRET    # or: run.bat --
join ... --secret ...
curl -fsSL "http://ADDRESS:7861/phone.sh?secret=SECRET" | sh             # in Termux on an
Android phone
python data/build_code.py --local C:/projects/app --github-repos 200 --langs
python,typescript
python tokenizer/train_tokenizer.py --code --raw data/code --vocab 16384 --out
tokenizer/bpe-code.json
python tokenizer/train_tokenizer.py --encode --raw data/code --tok tokenizer/bpe-code.json
--bin data/bin-code --fim-rate 0.5
python finetune/finetune_lora.py --base Qwen/Qwen2.5-Coder-0.5B-Instruct --code data/code
--examples 0 --seq 1024 --out runs/my-copilot
python data/code_update.py                          # automatic updates: one run with the
Copilot tab's settings
python data/code_update.py --install-schedule 24   # every 24 h, also when the Studio is
closed
```

See README.md for every option.

# 20. Troubleshooting

| Problem | What to do |
| --- | --- |
| The browser doesn't open | Open http://127.0.0.1:7860 yourself (or the port printed in the window). |
| Forgot the admin password | Stop the Studio and run `python -m studio.auth passwd <name>`. |
| Download keeps disconnecting (`WinError 10054`) | Press **Run** again; every source resumes. Under *Downloading* choose "Download whole files first" and more retries. |
| "Prepare the training data first" | Run Data steps 1–3 before training from scratch. |
| Resumed model writes nonsense after a corpus change | The tokenizer was rebuilt. Restore the old one (each run keeps a copy as `runs/<name>/tokenizer.json`, and the last replaced one is `tokenizer/bpe.prev.json`; copy it to `tokenizer/bpe.json`) and run Data step 3 again. Or use **Retrain** with corpus folders: it encodes them with the model's own tokenizer. |
| Retrain: "not this model's tokenizer" | The encoded corpus was made with a different tokenizer than the model. Untick *The whole encoded corpus* and tick the corpus folders instead. |
| Training on Intel Arc shows step 0 for minutes | The first step compiles the GPU kernels: wait up to 20 minutes (the log says "still on step 1"). On Arc graphics built into a laptop chip, the Small size is slow; the CPU size trains far faster. After the GPU start-up limit the run moves to the CPU by itself. |
| The GPU isn't used | Update the NVIDIA driver (Intel Arc: the Intel graphics driver), then `run.bat --reinstall` (or `--gpu` / `--xpu`). Check Settings → Hardware. On laptops, plug in the charger and set Windows Graphics to "High performance" for Python. |
| "CUDA out of memory" | Halve the batch size and double the gradient accumulation. |
| A PDF gives no text | It's a scan: install Tesseract OCR (see the README) or run `ocrmypdf` first. Password-protected: save it again without the password. The × line in the log says which. |
| A news source is "skipped until you confirm the rights" | Its license is copyrighted/unknown: tick that you have permission, or remove it. |
| Chat writes one sentence again and again | The model is small or not trained enough yet (see *How much training is enough* below). Meanwhile open Chat's settings and raise **Avoid repeats** to 1.3 and set **Never repeat a phrase** to 3. From-scratch models only continue text: use the **Short story** format, or start Free chat with the story's opening ("Once upon a time, a fox"). |

| Problem | What to do |
|---|---|
| A from-scratch story reads like a web page or an ad | The corpus is mostly web text (FineWeb) and the model is young. Train longer and give books and stories the biggest share of the data (*How much training is enough*). |
| Chat is slow with a fine-tuned model | Open the run and press **Make fast CPU copy**. |
| A worker keeps saying “coordinator not reachable” | Check the address and port, and that the coordinator’s firewall allows port 7861 (Windows asks once; allow it on private and public networks). From another network use the Internet address only when the card says “Reachable from other networks”; otherwise use Tailscale. The worker retries by itself. |
| “403 wrong secret” when joining | Copy the secret again from the coordinator’s Cluster page; a new secret is made each time a cluster starts unless you set one. |
| A GitHub repository was “skipped, licence …” | Only permissive licences are used (section 10). Pick another repository. |
| “GitHub rate limit reached; waiting” | Add a GitHub token on the Copilot tab, or let it wait; it continues by itself. |
| A project file is missing from the collected code | It is in a `.gitignore`, a skipped folder (`build`, `vendor`, …), too big, minified, generated, or not a known language. The log lists how many files each reason skipped. |
| Learning from news or ratings was “undone” | The quality gate saw the quick-check score drop; the previous model was kept. |

Only use texts you have the right to use. This project trains on public-domain and openly licensed text, plus your own writing.